

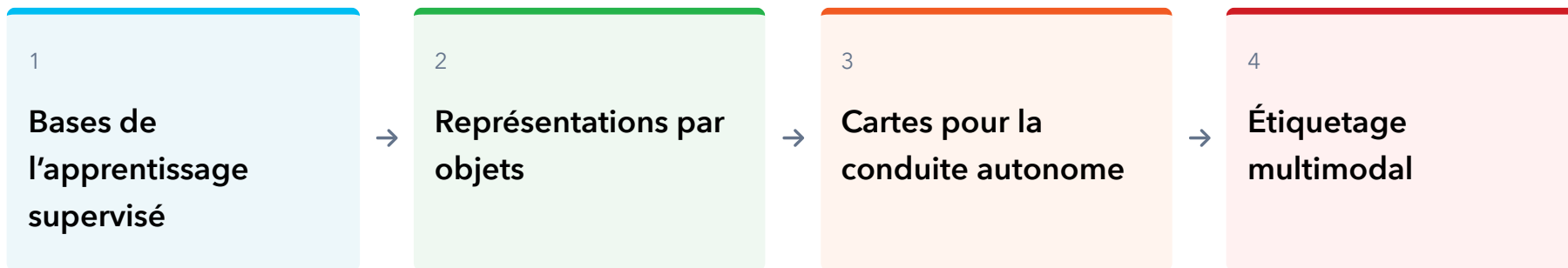
Apprentissage supervisé pour la perception robotique

INF8422 : Perception Robotique et Intelligence Spatiale

Prof. Pierre-Yves Lajoie · Polytechnique Montréal

**POLYTECHNIQUE
MONTREAL**

Apprendre une représentation utile au robot



Comprendre les données, les pertes et les calculs qui relient une observation à une représentation exploitable.

Objectif du cours

Pouvoir expliquer comment un modèle apprend, interpréter ses erreurs et intégrer ses sorties dans un système de perception géométrique utile pour la planification robotique.

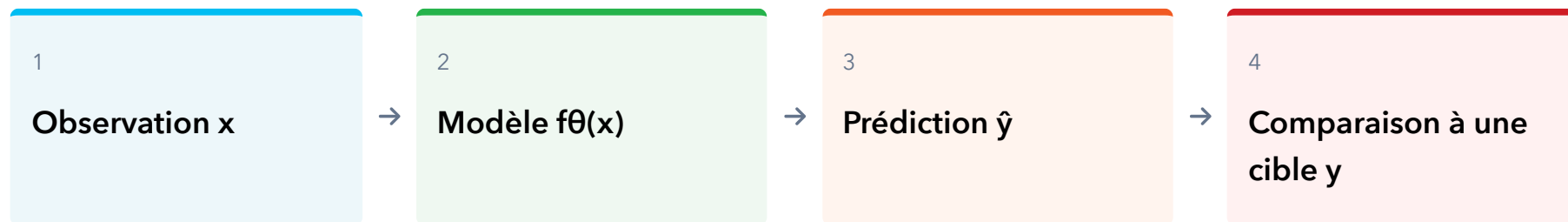
Plan

- Bases de l'apprentissage supervisé
- Représentations par objets
- Cartes pour la conduite autonome
- Supervision multimodale
- Synthèse

Bases de l'apprentissage supervisé

Comment transformer des exemples annotés en un modèle qui généralise ?

Une observation, une cible, une prédiction



Un robot reçoit une image. La cible peut être une **distance**, une **classe**, une **boîte** ou une **pose**.

Les paramètres θ sont ajustés sur des exemples annotés.

Exemple

Pour une distance réelle de 8 m, le modèle prédit 6 m. La fonction de perte quantifie cette erreur. L'erreur guide ensuite la correction des paramètres du modèle.

Apprendre en minimisant une fonction de coût

Pour un ensemble annoté $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} L(\theta), \quad L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$

1

Calculer $\hat{y}$

→

2

Mesurer ℓ versus la
cible y

→

3

Descente de
gradient et
rétropropagation
(backpropagation)

→

4

Le paramètres θ sont
modifiés

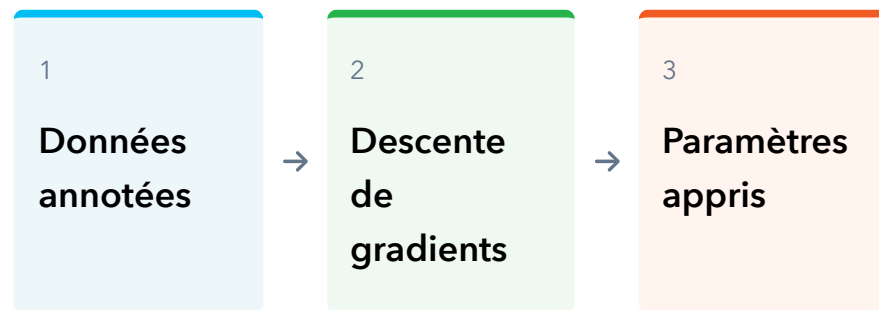
À retenir

Minimiser la fonction de perte $L(\theta)$ sur le jeu de données d'entraînement permet d'apprendre les paramètres θ de la fonction de prédiction f :

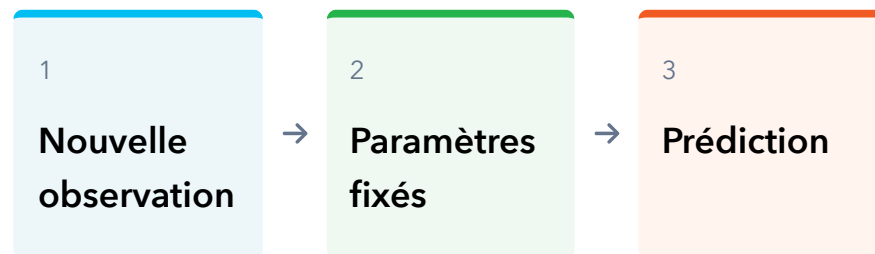
$$f_{\theta}(x_i) = \hat{y}_i$$

Entraîner puis utiliser le modèle

Apprendre un modèle:

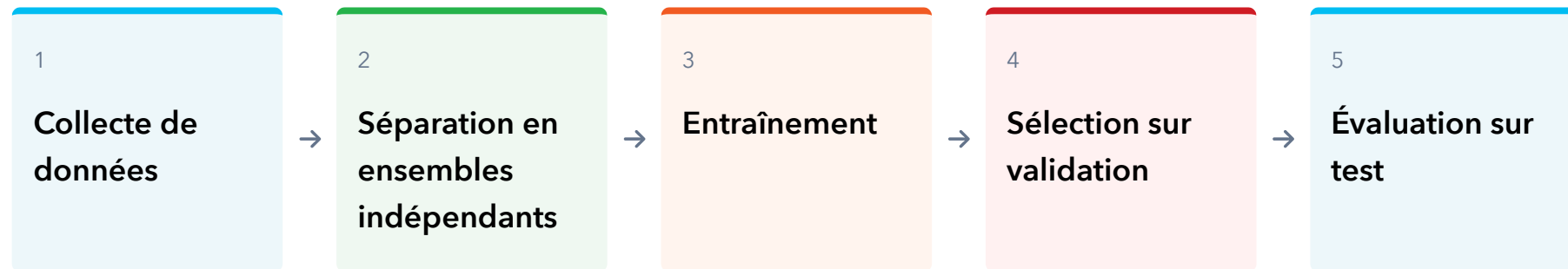


Utiliser un modèle:



Séparer entraînement, validation et test

Ensemble	Rôle	Ce qui peut en dépendre
Entraînement	Calculer la perte et les gradients	Paramètres du réseau
Validation	Comparer les choix de modèle	Hyperparamètres, arrêt, sélection
Test	Évaluer le modèle retenu	Rapport final de performance



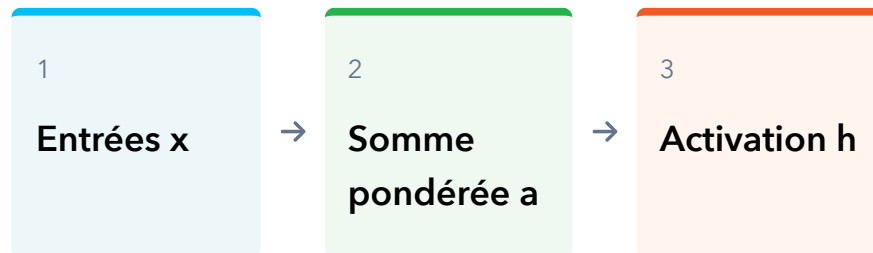
Éviter une contamination

En robotique, séparer par trajet, lieu ou session. Deux images successives d'une vidéo sont fortement corrélées.

Le neurone : une transformation suivie d'une activation

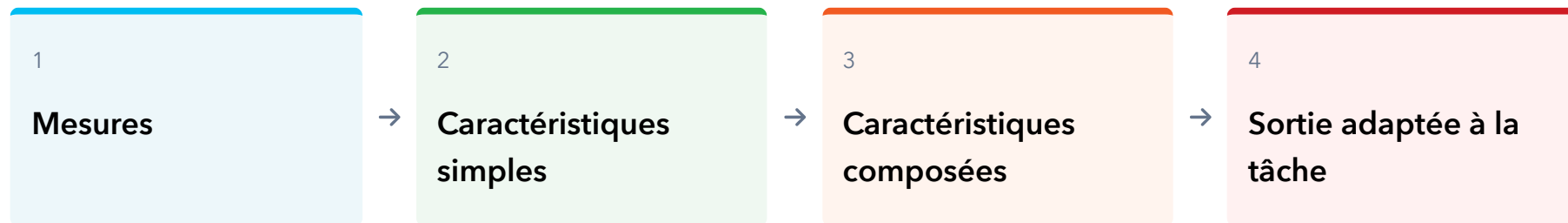
$$a = w^\top x + b, \quad h = \tanh(a).$$

- w pondère les entrées ; b est un biais.
- a est une préactivation.
- Des activations non-linéaires permettent d'apprendre des fonctions plus complexes.



Un réseau compose plusieurs transformations

$$h^{(1)} = \phi(W_1x + b_1), \quad h^{(2)} = \phi(W_2h^{(1)} + b_2), \quad \hat{y} = g(W_3h^{(2)} + b_3).$$



Les **poids** et **biais** constituent les paramètres. Une caractéristique intermédiaire peut décrire une texture, un contour ou une structure.

Un encodeur transforme une image ou un nuage en caractéristiques (features). La dernière couche d'un réseau (tête) estime une classe, une boîte, un descripteur, etc à partir des features.

Régression : prédire une quantité continue

La cible est numérique : profondeur, position, dimensions ou vitesse.

$$e = \hat{y} - y, \quad \ell_{\text{quad}} = \frac{1}{2}e^2, \quad \frac{\partial \ell}{\partial \hat{y}} = e.$$

Le gradient indique dans quel sens corriger la prédiction.

Exemple calculé

Distance vraie : 8 m. Prédiction : 6 m.

$$e = -2 \text{ m}, \ell = \frac{1}{2} \cdot (-2)^2 \text{ m}^2 \text{ et } \partial \ell / \partial \hat{y} = -2 \text{ m}.$$

Descendre le gradient fait **augmenter** la prédiction $\hat{y}$ pour réduire l'erreur.

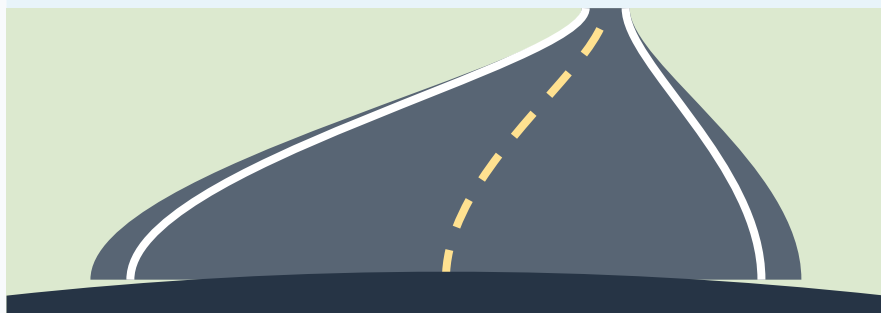
Exemple de régression : d'une image à un angle de conduite

Exemple de conduite par imitation · scènes et valeurs illustratives

Réinitialiser la démonstration

Entrée x · image de la caméra avant

Route, marquages et courbe à venir



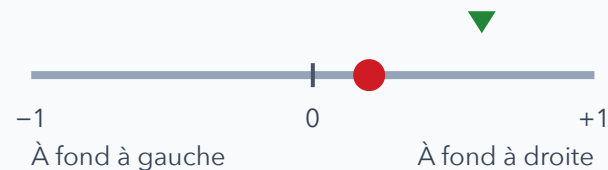
Virage à gauche

Route droite

Virage à droite

Sortie $\hat{y}$ · un seul nombre continu

$\hat{y} = 0,20$



Prédiction $\hat{y}$

▲ Cible enregistrée avec l'image : $y = 0,60$

Erreur $\hat{y} - y = -0,40$

Perte $\frac{1}{2}(\hat{y} - y)^2 = 0,080$

Une image 2D $\rightarrow$ un scalaire. Ici, l'angle est normalisé entre -1 et $+1$; il pourrait aussi être exprimé en radians.

Classification : des scores aux probabilités

Pour C classes, le réseau prédit des scores $s_1, \dots, s_C$ appelés logits :

$$p_c = \frac{\exp(s_c)}{\sum_{k=1}^C \exp(s_k)}, \quad \sum_c p_c = 1.$$



Classer une région observée par le robot

Dans l'ordre **route, piéton, véhicule**, ces scores donnent 66,5 %, 24,5 % et 9,0 %. Le modèle choisit « route ». Si l'annotation est « piéton », cette prédiction est incorrecte.

À retenir

Les scores peuvent être négatifs. Les probabilités sont positives et normalisées. Le score le plus grand reste la classe la plus probable.

Entropie croisée : la probabilité de la bonne classe

Une région contient un **piéton**. Quelle probabilité le modèle attribue-t-il à cette annotation ?

Classe c	Cible y_c	Prédiction p_c
Route	0	0,6652
Piéton	1	0,2447
Véhicule	0	0,0900

L'étiquette **one-hot** vaut 1 pour la bonne classe, 0 pour les autres. Les probabilités p_c viennent du softmax.

L'entropie croisée compare la cible y à la distribution prédite p :

$$\ell_{\text{CE}} = - \sum_{c=1}^C y_c \ln p_c.$$

Seul le terme de la classe annotée c^* reste :

$$\ell_{\text{CE}} = - \ln p_{c^*}.$$

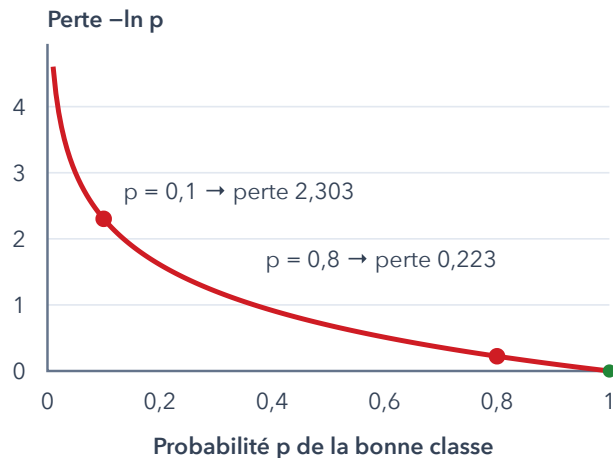
La cible sélectionne le terme « piéton »

$$\ell = -[0 \ln p_{\text{route}} + 1 \ln p_{\text{piéton}} + 0 \ln p_{\text{véhicule}}] = -\ln(0,2447) \approx 1,408.$$

Minimiser cette perte encourage le modèle à attribuer **plus de probabilité au piéton**.

Pourquoi une perte logarithmique ?

Avec $p = p_{c^*}$, la perte $-\ln p$ diminue lorsque la probabilité de la bonne classe augmente.



Deux prédictions correctes, deux pertes

La cible est « piéton » ; ordre : route, piéton, véhicule.

Probabilités prédites	Perte
(0,4; 0,5; 0,1)	0,693
(0,05; 0,9; 0,05)	0,105

Le piéton arrive premier dans les deux cas, mais la seconde prédiction lui donne plus de probabilité.

À $p = 1$: perte nulle. Si $p \rightarrow 0$: perte $\rightarrow +\infty$.

Ce que l'on entraîne

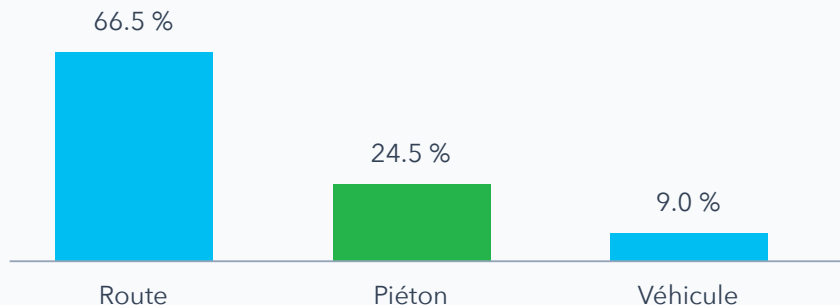
La perte évalue la **probabilité de l'annotation**, au-delà du simple verdict correct / incorrect. Sur un mini-lot (batch), on prend la moyenne des pertes des exemples.

Manipuler les scores et l'entropie croisée

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

Probabilités : somme = 1



Cible : **Piéton**

Étiquette $y = [0 ; 1 ; 0]$

$p(\text{cible}) = \mathbf{0.2447}$

$\ell = -\ln p(\text{cible}) = \mathbf{1.408}$

Gradients des scores : $p - y$
0.665 ; -0.755 ; 0.090

Score Route : 2.0



Score Piéton : 1.0



Score Véhicule : 0.0



Piéton ▼

Cible « piéton » : augmenter son score fait baisser la perte. Changer ensuite la cible pour comparer.

La règle de chaîne relie la perte aux paramètres

Modifier un poids change une activation, puis la prédiction, puis la perte. La règle de chaîne mesure **comment cet effet se transmet**.

$$a = w_1x + b_1, \quad h = \tanh(a), \quad \hat{y} = w_2h + b_2, \quad \ell = \frac{1}{2}(\hat{y} - y)^2.$$

$$\frac{\partial \ell}{\partial w_1} = (\hat{y} - y) \times w_2 \times (1 - h^2) \times x.$$

$\hat{y} - y$	w_2	$1 - h^2$	x
Pente de la perte	Effet de h sur $\hat{y}$	Pente de l'activation	Effet de w_1 sur a

Rétropropager

Le calcul avant fournit $a, h, \hat{y}$. Le calcul arrière multiplie les dérivées locales pour obtenir la sensibilité de la perte à chaque paramètre.

Rétropropagation : un poids, un calcul, un effet

Exemple de braquage : $x = 1, y = 0,3$, avec $w_1 = b_1 = 0, w_2 = 0,5$ et $b_2 = 0,1$.

Calcul avant	Valeur	Dérivée locale pour le retour
$a = w_1x + b_1$	0	$\partial a / \partial w_1 = x = 1$
$h = \tanh(a)$	0	$\partial h / \partial a = 1 - h^2 = 1$
$\hat{y} = w_2h + b_2$	0,1	$\partial \hat{y} / \partial h = w_2 = 0,5$
$\ell = \frac{1}{2}(\hat{y} - y)^2$	0,02	$\partial \ell / \partial \hat{y} = -0,2$

Quel poids faut-il augmenter ?

$$\partial \ell / \partial w_1 = (-0,2) \times 0,5 \times 1 \times 1 = -0,1.$$

Avec $\eta = 0,2$, $w_1' = 0 - 0,2(-0,1) = 0,02$. En changeant seulement ce poids, $\hat{y}' \approx 0,110$: l'angle se rapproche de la cible 0,3.

Descente de gradient : choisir une direction et un pas

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k).$$

- Le gradient pointe vers la hausse locale la plus rapide.
- Le signe moins donne une direction de descente.
- $\eta > 0$ contrôle la longueur du pas.

Un pas numérique

$$L(w) = \frac{1}{2}(w - 3)^2.$$

$$\text{À } w = 1, \nabla L = -2.$$

$$\text{Avec } \eta = 0,2 : w' = 1 - 0,2(-2) = 1,4.$$

La perte passe de 2 à 1,28.

Lot complet, SGD et mini-lots

Un mini-lot B contient $|B|$ exemples. On **moyenne leurs gradients**, puis on effectue une mise à jour :

$$g_B = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i, \quad \theta \leftarrow \theta - \eta g_B.$$

Méthode	Exemples par mise à jour	Pour 3 200 images, en une époque
Lot complet	Tous les exemples	1 mise à jour
SGD, un exemple à la fois	1	3 200 mises à jour
Mini-lot	32 dans cet exemple	100 mises à jour

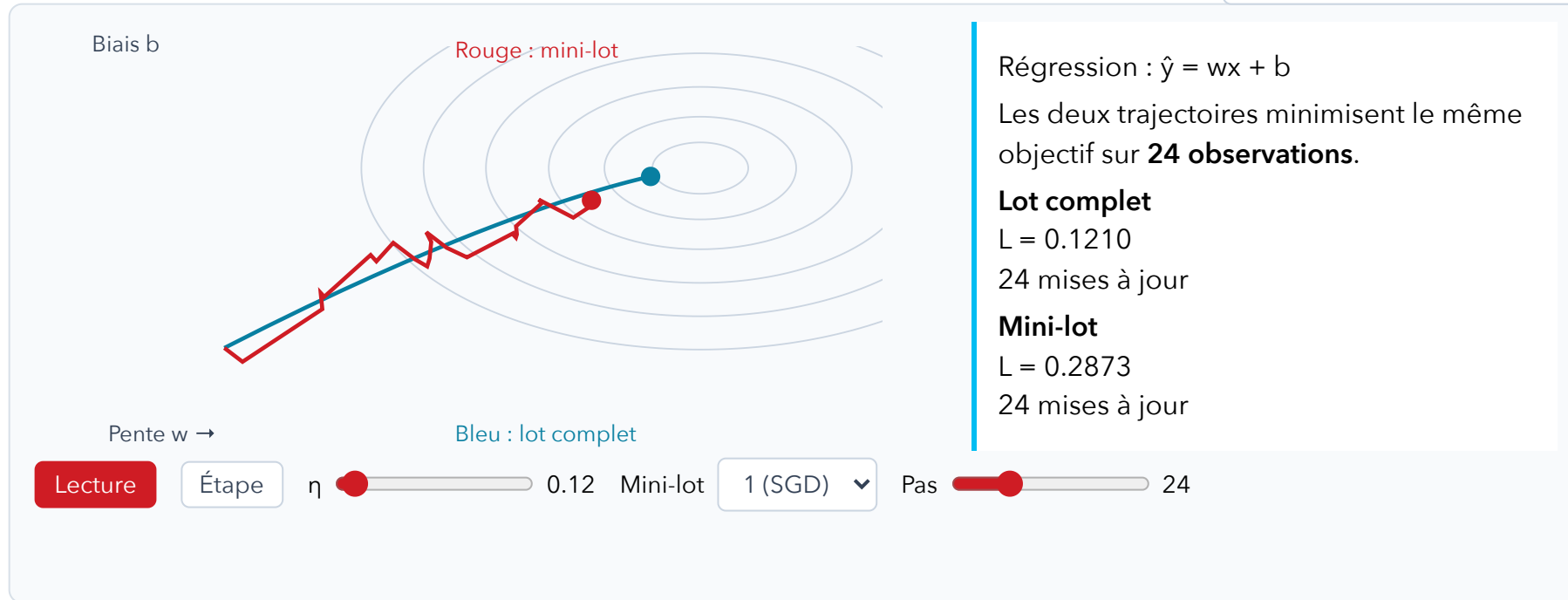
Deux exemples ne poussent pas toujours dans le même sens

Pour un même poids, deux images donnent les gradients -2 et $+1$. Le mini-lot donne $g_B = (-2 + 1)/2 = -0,5$: avec $\eta = 0,2$, le poids augmente de $0,1$.

Comparer les trajectoires d'optimisation

Expérience pédagogique · données synthétiques

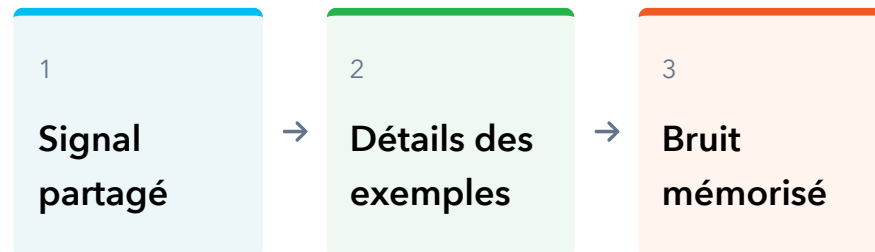
Réinitialiser la démonstration



Même objectif et même départ. Modifier η puis la taille du mini-lot ; observer stabilité, bruit et vitesse.

Quand apprendre davantage dégrade la généralisation

Un modèle très flexible peut apprendre des détails propres aux exemples et au bruit d'annotation. Sa perte d'entraînement continue alors à baisser, tandis que sa perte de validation augmente.



Surapprentissage

Le surapprentissage se diagnostique par l'écart de généralisation, pas uniquement par une faible perte d'entraînement.

Surapprendre c'est comme surmémoriser

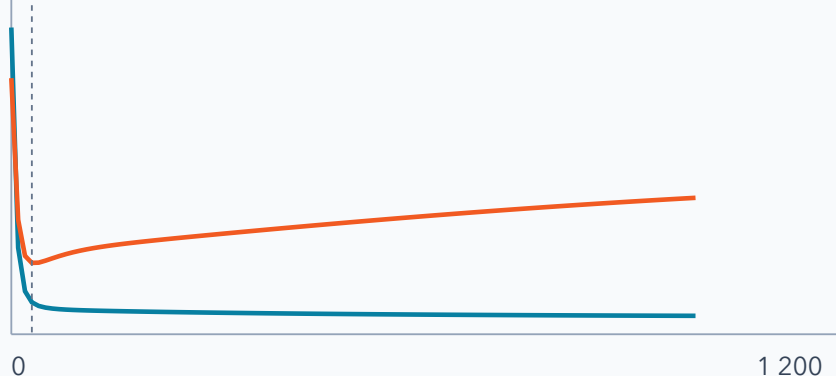
Le modèle associe un bâtiment rouge à un virage à droite. Il prédit bien les angles sur ce trajet, mais se trompe sur une route inconnue où un bâtiment rouge précède un virage à gauche.

Choisir le modèle avec la validation

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

Erreur quadratique moyenne
Meilleure validation



Entraînement : 0.034

Validation : 0.249

Test masqué pendant le choix du modèle.

Arrêt conseillé : époque 30.

Capacité 16 Époques 1000
Bruit 0.55 λ 0.000

Explorer capacité, bruit et régularisation. Choisir une époque, puis figer le modèle avant de révéler le test.

Régulariser : limiter les solutions trop fragiles

Le but de la régularisation est de prévenir le surapprentissage et assurer la généralisation.

Exemple de régularisation: Pénalisation L2 sur les poids du réseau :

$$L_{\text{reg}}(\theta) = \underbrace{\frac{1}{N} \sum_i \ell_i}_{\text{ajustement aux données}} + \underbrace{\lambda \sum_j \theta_j^2}_{\text{pénalité sur les poids}} .$$

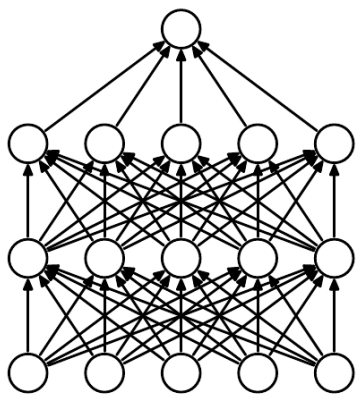
Deux solutions donnent la **même perte de données** 0,10. Avec $\lambda = 0,01$:

Poids	$\ \theta\ ^2$	Pénalité	Coût total
$\theta_A = (3, 4)$	25	0,25	0,35
$\theta_B = (1, 2)$	5	0,05	0,15

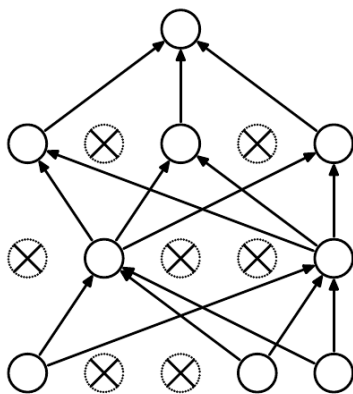
Une préférence, à régler sur la validation

L'objectif préfère ici B. Un λ trop grand peut toutefois empêcher d'apprendre le signal : une faible norme ne garantit pas une bonne généralisation.

Régularisation Dropout : apprendre avec des sous-réseaux



(a) Standard Neural Net



(b) After applying dropout.

À l'entraînement, chaque activation est conservée avec probabilité $1 - p$.

$$\tilde{h} = \frac{mh}{1-p}, \quad m \sim \text{Bernoulli}(1-p).$$

À l'inférence, généralement, toutes les activations sont utilisées : $\tilde{h} = h$.

Dropout réduit le nombre d'activations et de paramètres affectés lors de l'entraînement. Règle générale: Moins de paramètres pour la même quantité de données réduit les risques de surapprentissage.

Srivastava et al., JMLR 2014, Figure 1. [Article et source de la figure.](#)

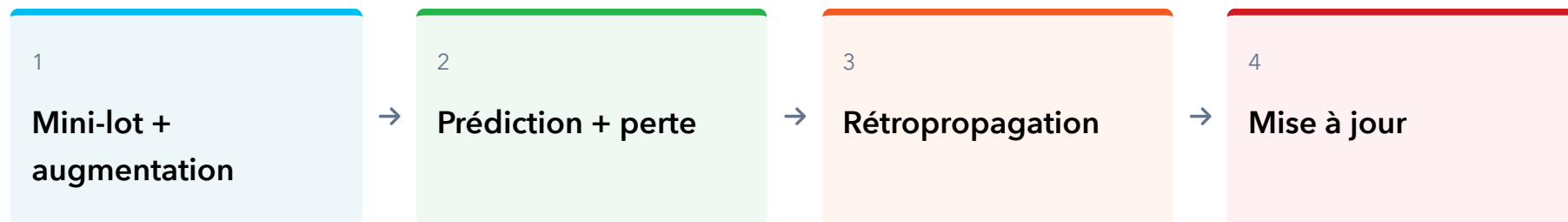
Augmenter des données de robotique

Observation	Transformations utiles	Cohérence à préserver
Image couleur	Éclairage, contraste, flou, occultation	Identité de la classe et limites physiques
Image + boîtes	Recadrage, translation, redimensionnement	Boîtes et masques transformés ensemble
RGB-D / multivue	Transformations compatibles entre capteurs	Profondeur, intrinsèques, correspondances
Nuage LiDAR	Rotation autour de la verticale, bruit, retrait de points	Boîtes et poses dans le même repère

Validité des exemples

Une augmentation doit représenter une variation plausible de la tâche. Une rotation arbitraire de la gravité peut produire une scène impossible et nuire à l'entraînement.

Une boucle d'apprentissage complète



```
for x, y in training_loader:
    x, y = augment_together(x, y)
    optimizer.zero_grad()
    prediction = model(x)
    loss = criterion(prediction, y)
    loss.backward()
    optimizer.step()
```

Exemple image-angle : un mini-lot contient 32 images RVB ($x : 32 \times 3 \times H \times W$) et 32 angles ($y : 32 \times 1$). Le modèle prédit un angle par image.

Validation en mode évaluation, sans gradients ; sauvegarder le meilleur modèle selon le critère choisi.

Représentations par objets

Détecter, localiser, orienter et organiser les objets d'une scène.

Que doit contenir une représentation d'objet ?

Sortie	Information	Usage robotique
Classe + boîte 2D	Catégorie et région dans l'image	Sélection de régions, suivi visuel
Boîte 3D	Centre, dimensions, orientation	Distance, encombrement, collision
Pose d'un objet connu	Transformation du modèle vers le capteur	Préhension, assemblage, inspection
Nœud dans une carte	Identité persistante et relations	Recherche d'objets, navigation sémantique



Détection 2D : les données d'entraînement

Une image peut contenir un nombre variable d'objets :

$$Y = \{(c_j, b_j)\}_{j=1}^M, \quad b_j = (u_j, v_j, w_j, h_j).$$

Pour chaque objet, c_j est la classe ; (u_j, v_j) est le **coin supérieur gauche** et (w_j, h_j) la largeur et la hauteur, en pixels.

Exemple

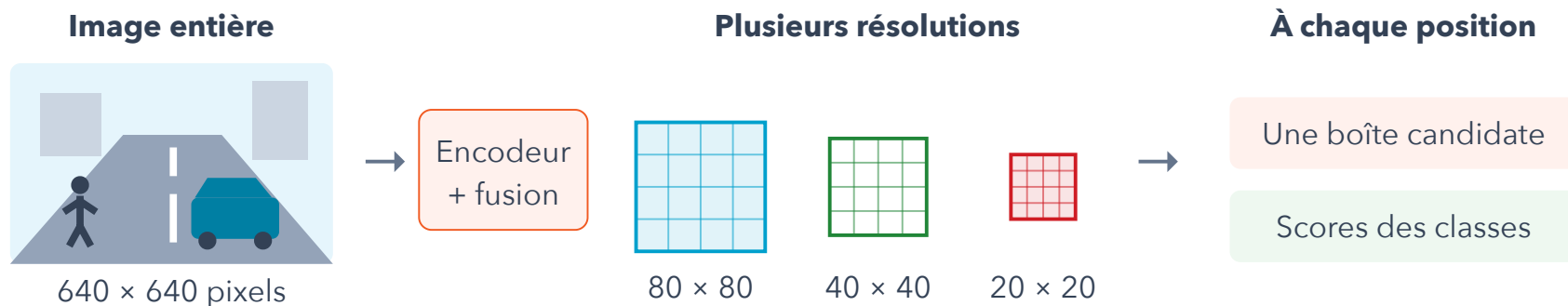
Exemple : une image annotée avec deux voitures et un piéton. Le détecteur doit prédire leurs classes, leurs positions et leurs dimensions dans l'image.



YOLO : détecter en une seule passe

YOLO – You Only Look Once prédit les classes et les boîtes à partir de l'image entière, en une passe du réseau.

Exemple présenté : **YOLOv8**.

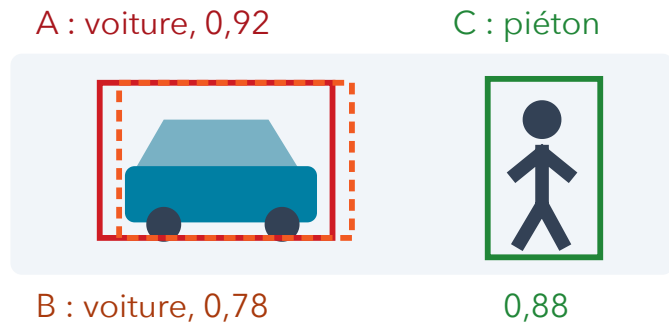


À 640 × 640 pixels, les trois cartes donnent $80^2 + 40^2 + 20^2 = 8\,400$ positions de prédiction. Les résolutions différentes aident à détecter des objets de tailles variées.

Redmon et al., YOLO, CVPR 2016 ; exemple moderne : Ultralytics YOLOv8. Schéma pédagogique.

YOLO : garder les détections utiles

À l'inférence, YOLOv8 peut proposer plusieurs boîtes pour un même objet. Deux étapes réduisent ces candidats : **seuil de score**, puis **suppression des doublons (NMS)**.



Seuils : score 0,50 ; IoU 0,50.

Candidat	Classe	Score
A	Voiture	0,92
B	Voiture	0,78
C	Piéton	0,88
D, non dessiné	Voiture	0,20

La **NMS** conserve la boîte de meilleur score, puis écarte les boîtes de même classe qui la recouvrent trop. Elle répète ce choix parmi les candidates restantes.

Quelles boîtes conserver ?

D est écartée par le seuil de score. Comme $\text{IoU}(A, B) \approx 0,85 > 0,50$, A est gardée et B supprimée. Il reste **A et C**.

NMS : non-maximum suppression ; IoU : intersection sur union. Post-traitement de YOLOv8. Les seuils sont pédagogiques.

Une perte pour la classe, une perte pour la boîte

Une fois chaque prédiction associée à sa cible, on corrige **ce qu'est l'objet** et **où il se trouve** :

$$L_{\text{det}} = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{box}} L_{\text{box}} + \lambda_{\text{IoU}} L_{\text{IoU}}.$$

Terme	Ce qu'il compare	Exemple d'erreur
Classification	Probabilités et classe cible	Une voiture classée piéton
Régression de boîte	Coordonnées prédites et annotées	Une boîte décalée de 40 pixels
Recouvrement	Régions couvertes par les boîtes	Une boîte trop grande malgré un bon centre

Les poids λ équilibrent les contributions

Les termes ont des échelles différentes. Leur pondération et leur forme exacte dépendent du détecteur. Une boîte parfaite peut encore avoir une mauvaise classe.

IoU : combien de surface les boîtes partagent-elles ?

L'intersection sur union compare la partie commune à toute la région couverte :

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

Deux boîtes de 100×100 pixels, à la même hauteur, décalées horizontalement de 40 pixels :

Quantité	Calcul	Aire en pixels carrés
Intersection	$(100 - 40) \times 100$	6 000
Union	$10\,000 + 10\,000 - 6\,000$	14 000

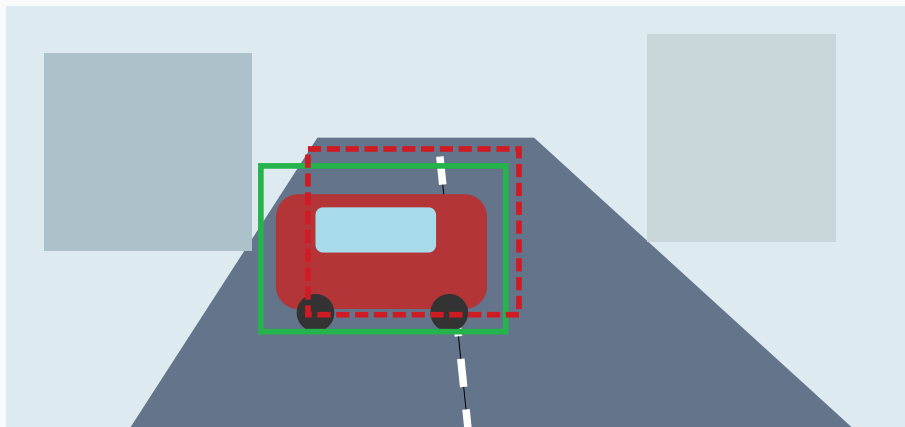
Un recouvrement visible peut rester insuffisant

$\text{IoU} = 6\,000 / 14\,000 \approx 0,43$. À un seuil d'évaluation de 0,50, cette localisation ne suffit pas, même si la classe est correcte.

Visualisation IoU

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration



Vert : annotation

Rouge : prédiction

x y Largeur Score

Classe cible : **voiture**

$p(\text{voiture}) = 0.818$

Intersection / union :

IoU = 0.638

$\ell_{\text{classe}} = 0.201$

$\ell_{\text{boîte}} = 0.051$

Somme illustrative : **0.253**

Faire coïncider les boîtes, puis rendre la classe incorrecte : les deux erreurs se contrôlent séparément.

Évaluer les détections au bon niveau

Une détection correcte combine une classe correcte et un recouvrement suffisant avec une annotation. Une même annotation ne doit pas valider plusieurs prédictions.

$$\text{précision} = \frac{VP}{VP + FP}, \quad \text{rappel} = \frac{VP}{VP + FN}.$$

VP : vrais positifs ; FP : faux positifs ; FN : faux négatifs.

Du score à la performance

La précision moyenne, AP, résume la courbe précision-rappel. Toujours préciser les classes et seuils de recouvrement du protocole.

Trois piétons annotés, quatre détections

Deux piétons sont correctement détectés, un est manqué ; une boîte est un doublon et une autre vise un poteau. Donc $VP = 2$, $FP = 2$, $FN = 1$: **précision = 50 %**, **rappel $\approx$ 67 %**.

Du seuil de confiance à la courbe précision-rappel

Pour une classe et un seuil IoU fixés, trier les détections par score décroissant. Exemple : **trois piétons annotés**.

Score	Détection ajoutée	Précision cumulée	Rappel cumulé
0,95	Piéton 1 : VP	$1/1 = 100\%$	$1/3$
0,90	Poteau : FP	$1/2 = 50\%$	$1/3$
0,80	Piéton 2 : VP	$2/3 \approx 67\%$	$2/3$
0,60	Doublon du piéton 1 : FP	$2/4 = 50\%$	$2/3$
0,40	Piéton 3 : VP	$3/5 = 60\%$	$3/3$

Abaisser le seuil fait entrer davantage de détections. Le rappel augmente ou reste constant ; la précision peut monter ou descendre.

AP résume la courbe précision-rappel selon le protocole d'interpolation. Ce n'est pas la moyenne des cinq précisions du tableau.

Exemple fictif, annotations sans cas « crowd » ni ignorés. Protocole de référence COCO.

Faire varier le seuil de détection

Image synthétique · scores fixes · classe : piéton · sans NMS

Réinitialiser la démonstration

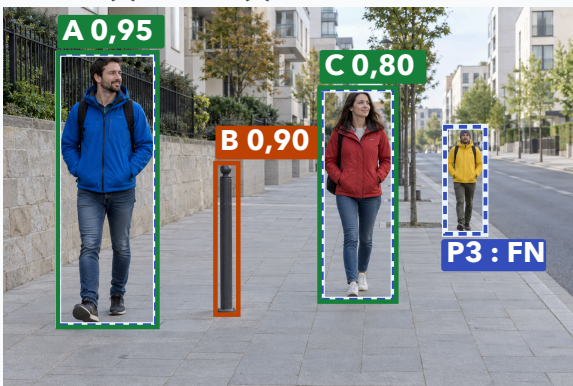
Seuil de score τ  0,80

Balayer le seuil

☒ Annotations

IoU d'évaluation $\geq 0,50$

Image : 3 détection(s) retenue(s)



 VP : piéton retrouvé  FP : erreur ou doublon  Pointillés bleus : annotations Courbe brute · lettres = dernières boîtes ajoutées

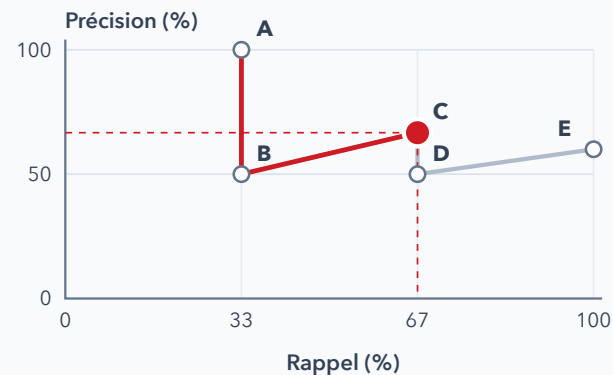
VP 2 · FP 1 · FN 1

Précision : $2/3 = 66,7 \%$

Rappel : $2/3 = 66,7 \%$

C retrouve un autre piéton : le rappel et la précision augmentent.

Courbe précision-rappel : point au seuil courant



Abaisser le seuil ou cliquer sur la courbe. Entre deux scores, les détections et le point restent identiques.

RGB-D : passer de l'image à des points métriques

Le pixel (u, v) et sa profondeur axiale $d = D(u, v)$ donnent un point dans le repère caméra :

$$X_C = \begin{bmatrix} (u - c_x)d/f_x \\ (v - c_y)d/f_y \\ d \end{bmatrix} = dK^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}.$$

Soustraire le point principal (c_x, c_y) , diviser par les focales (f_x, f_y) , puis multiplier par la profondeur : les pixels deviennent des mètres.

Un point sur une boîte à saisir

$f_x = f_y = 500$ px, $(c_x, c_y) = (320, 240)$, $(u, v) = (370, 240)$ et $d = 2$ m donnent $X_C = (0,20 ; 0 ; 2)$ m : **20 cm à droite** de l'axe optique.

Pour le véhicule : $X_V = R_{V \leftarrow C} X_C + t_{V \leftarrow C}$. La calibration extrinsèque fixe ce changement de repère.

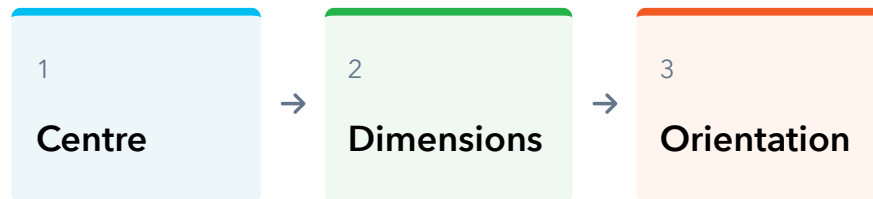
Paramétrer une boîte 3D

Pour des objets supposés verticaux :

$$b = (c_x, c_y, c_z, l, w, h, \psi).$$

- c : centre, en mètres.
- l, w, h : dimensions.
- ψ : orientation autour de la verticale.

Une orientation générale nécessite une rotation 3D.



Hypothèse physique

Une boîte de véhicule ne nécessite qu'un seul angle lacet (yaw), puisqu'on assume que la voiture a ses quatre roues au sol. Par contre, un outil pouvant être tenu dans toutes les orientations demande trois degrés de liberté de rotation.

Une détection LiDAR versus caméra



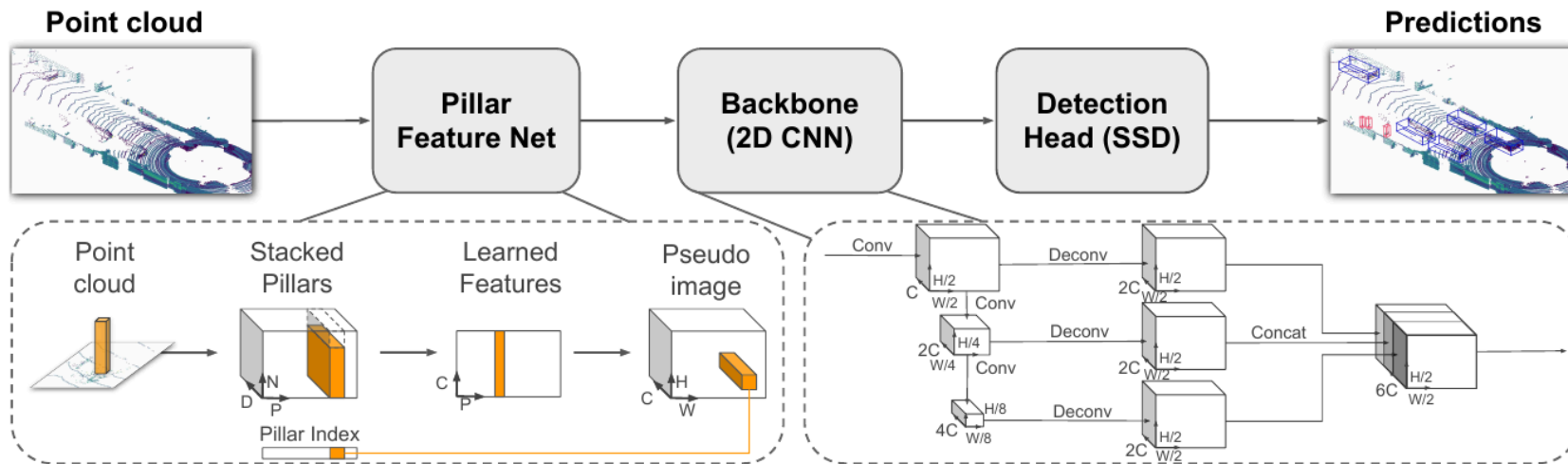
En haut : nuages de points et boîtes en vue de dessus.

En bas : reprojektion des boîtes dans les images.

Les images aident ici à **visualiser** le résultat ; PointPillars utilise le LiDAR pour cette détection.

Lang et al., PointPillars, CVPR 2019, Figure 3. [Article et source de la figure.](#)

PointPillars : du nuage à une pseudo-image



1. Regrouper les points en colonnes verticales.
2. Encoder les points de chaque colonne.
3. Organiser les vecteurs dans une grille.
4. Prédire classes et boîtes 3D.

La grille facilite le traitement par des convolutions 2D.

Lang et al., PointPillars, CVPR 2019, Figure 2. [Article et source de la figure.](#)

Images et LiDAR : des informations complémentaires

Critère	Images	LiDAR
Apparence et catégories	Texture et couleur riches	Géométrie, intensité éventuelle
Échelle métrique	Ambiguë en monoculaire	Mesure directe de distance
Densité	Dense en pixels	Clairsemée, surtout à longue portée
Conditions difficiles	Sensible à l'éclairage	Sensible aussi à la pluie, au brouillard et aux occultations
Fusion	Calibration et synchronisation nécessaires	Même exigence pour une fusion cohérente

Décision de conception

Choisir selon le capteur, la tâche et les conditions de déploiement. Une fusion mal calibrée peut dégrader une estimation.

Un piéton attend au bord de la route

L'image aide à reconnaître le piéton ; les retours LiDAR sur son corps renseignent sa distance. La fusion doit associer les pixels et les points du même piéton, au même instant.

Estimer la pose d'un objet connu

Pour plusieurs applications, il est nécessaire d'estimer la pose d'un objet.

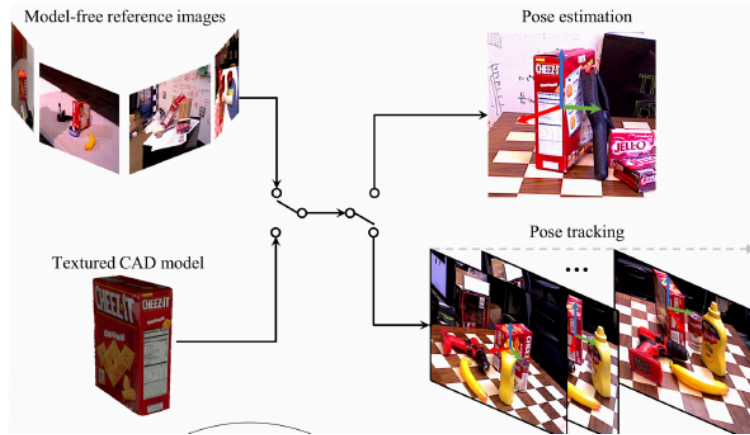
Le cas le plus simple est lorsqu'on connaît à priori la géométrie de l'objet. Par exemple, on peut avoir un modèle CAO (CAD en anglais) de l'objet.

Le modèle CAO définit les points sur l'objet dans le repère objet O . La pose les exprime dans le repère caméra C :

$$X_C = \underbrace{R_{C \leftarrow O} X_O}_{\text{tourner les axes de l'objet}} + \underbrace{t_{C \leftarrow O}}_{\text{placer son origine}} .$$

Six degrés de liberté : trois pour la translation et trois pour la rotation. Les dimensions de l'objet sont fournies par son modèle.

Un modèle CAO fourni à l'inférence



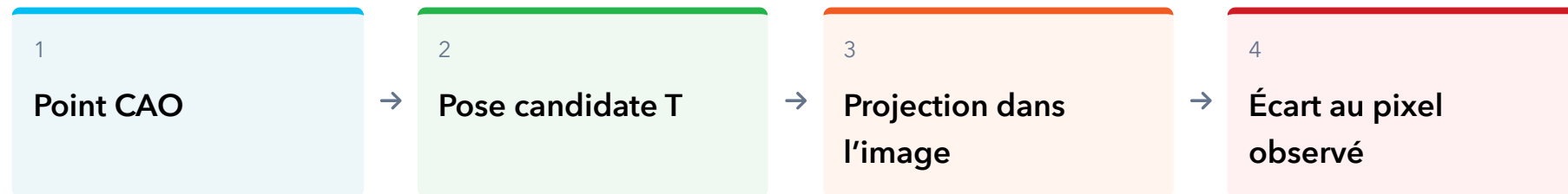
1. On cherche l'objet dans l'image et détermine sa position et boîte englobante.
2. Pour obtenir l'orientation de l'objet, on associe les points du modèle 3D (CAO) avec les points dans la boîte englobante.
3. Une fois les points associés, on peut calculer l'orientation (ex: à la manière d'ICP et PnP).

Wen et al., FoundationPose, CVPR 2024, Figure 1, partie supérieure. [Article et source de la figure.](#)

Correspondances puis estimation géométrique

Pour chaque point, on connaît sa position **3D dans le modèle** X_k^O et son **pixel observé** u_k . La pose T reste à estimer :

$$\hat{T} = \arg \min_T \sum_k \left\| u_k - \underbrace{\pi(KTX_k^O)}_{\text{pixel prédit pour cette pose}} \right\|^2.$$



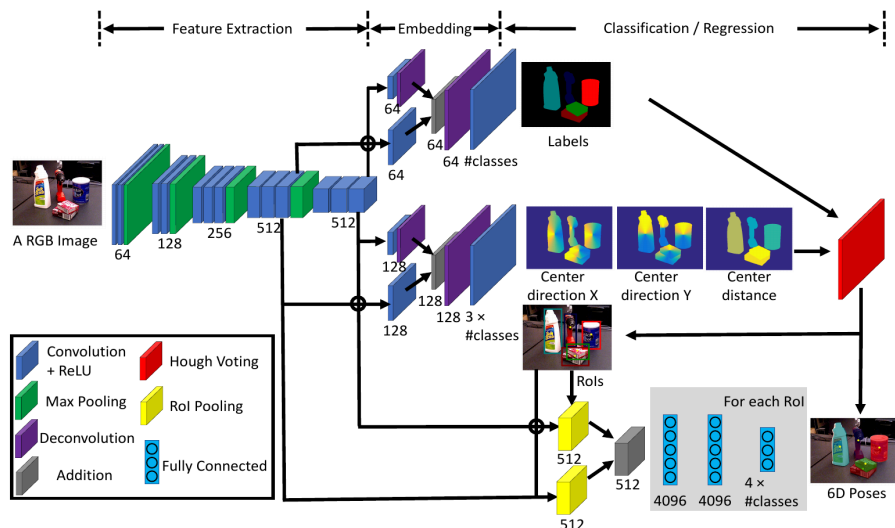
Le résidu est mesuré dans l'image

Un coin observé en (320, 240) est projeté en (324, 237) : le résidu vaut (−4, 3) pixels.

Sa contribution à la somme est $(-4)^2 + 3^2 = 25 \text{ px}^2$.

PnP estime la pose à partir de ces correspondances ; un raffinement réduit l'erreur de reprojection.

PoseCNN : prédire la pose avec des exemples annotés



Des caractéristiques (features) partagées alimentent trois sorties : segmentation, translation et rotation.

Les cibles d'entraînement contiennent des poses connues. La rotation est régressée sous forme de quaternion.

Le modèle est spécialisé pour les objets appris.

Xiang et al., PoseCNN, RSS 2018, Figure 2. [Article et source de la figure.](#)

Quaternions : normalisation et double représentation

Un quaternion unitaire $q = (q_w, q_x, q_y, q_z)$ représente une rotation :

$$\|q\|_2 = 1, \quad R(q) = R(-q).$$

Normaliser la sortie du réseau avant de former la matrice de rotation.

Ambiguïté de signe

Une perte naïve $\|\hat{q} - q\|^2$ peut pénaliser deux rotations identiques.

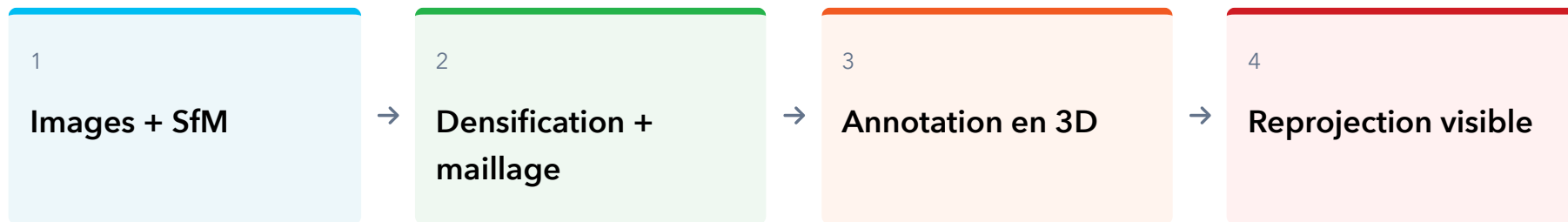
Une possibilité :

$$\min (\|\hat{q} - q\|^2, \|\hat{q} + q\|^2) .$$

Même orientation, erreur artificielle

$q = (1, 0, 0, 0)$ et $\hat{q} = (-1, 0, 0, 0)$ laissent tous deux l'objet sans rotation. Pourtant $\|\hat{q} - q\|^2 = 4$; la perte qui tient compte du signe vaut 0.

Annoter une reconstruction pour superviser plusieurs vues

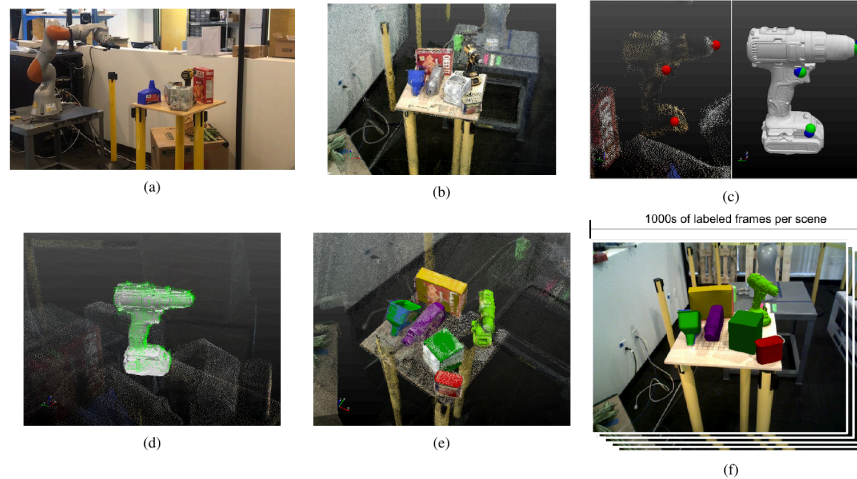


La structure à partir du mouvement, **SfM**, estime les caméras et une géométrie à partir d'images. **SfM** est l'équivalent du SLAM, mais hors ligne, donc sans contrainte d'exécution temps réel. Une reconstruction dense et un maillage permettent ensuite une annotation spatiale.

Économiser l'annotation

Une annotation 3D peut servir à de nombreuses images. Il faut des poses cohérentes, une scène suffisamment statique et un traitement des occultations.

Annoter une scène reconstruite



Le pipeline collecte une séquence RGB-D, reconstruit la scène, aligne les modèles d'objets, puis reprojette leurs étiquettes.

Marion et al., LabelFusion, ICRA 2018, Figure 1. [Article et source de la figure.](#)

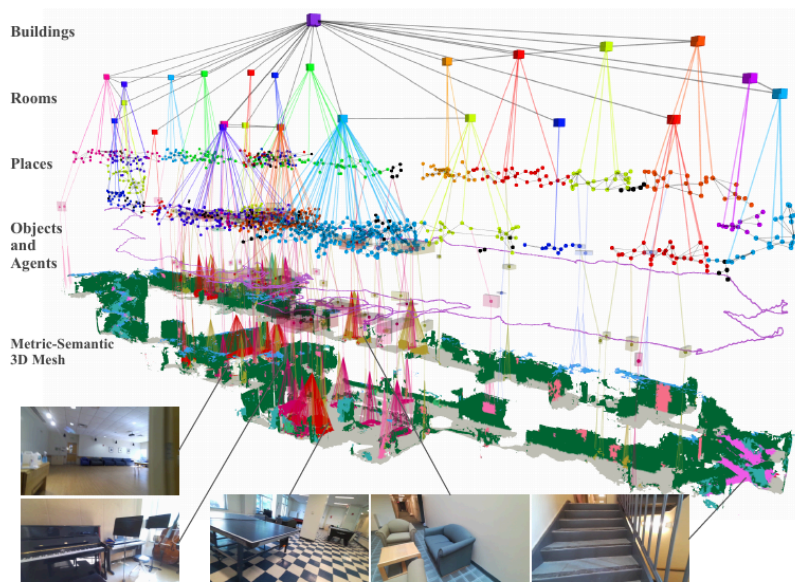
Des objets à une carte hiérarchique

Graphe de scène 3D: organise géométrie, objets, lieux, pièces et bâtiment dans une même hiérarchie.

Les noeuds (objets, lieux, pièces) ont des étiquettes de classe et de pose.

Les relations représentent l'appartenance, la proximité et la connectivité.

Exemple : « Trouver une chaise dans la salle de réunion. » Le graphe relie la chaise à la pièce, puis la pièce aux lieux traversables pour y accéder.

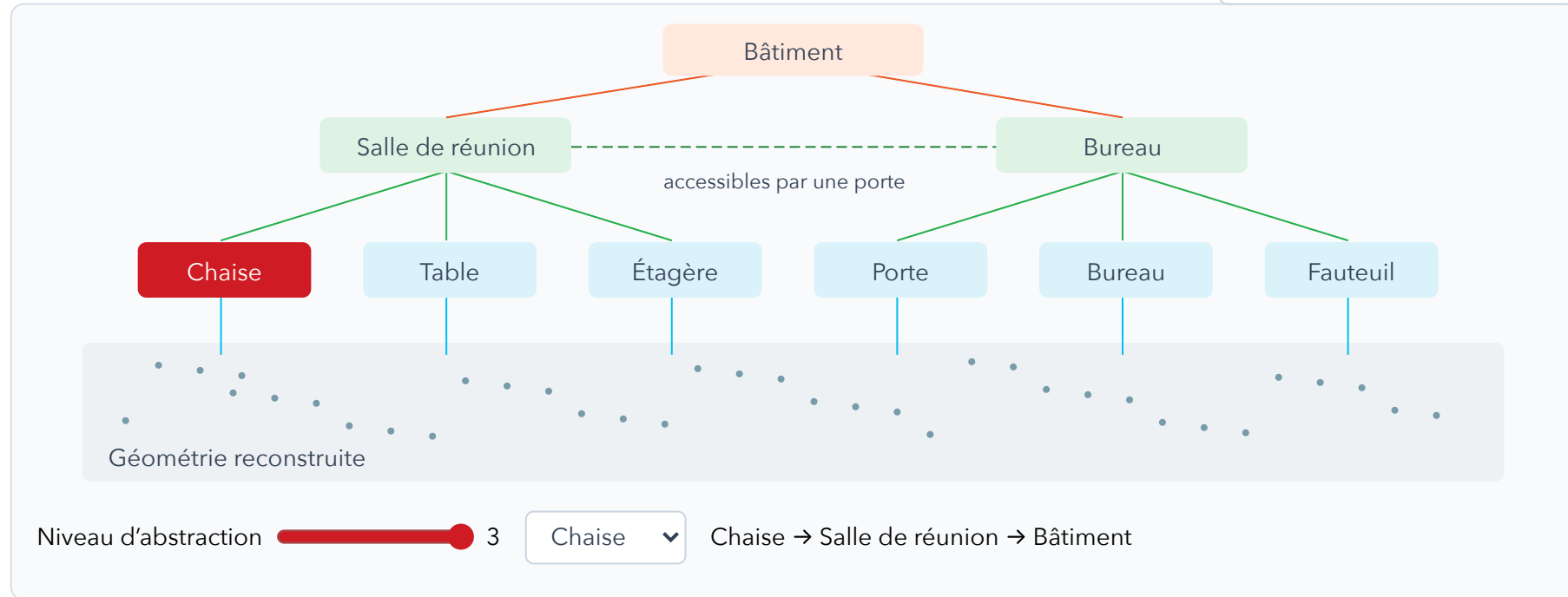


Hughes et al., Hydra, RSS 2022, Figure 1. [Article et source de la figure.](#)

Explorer les niveaux d'un graphe de scène

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

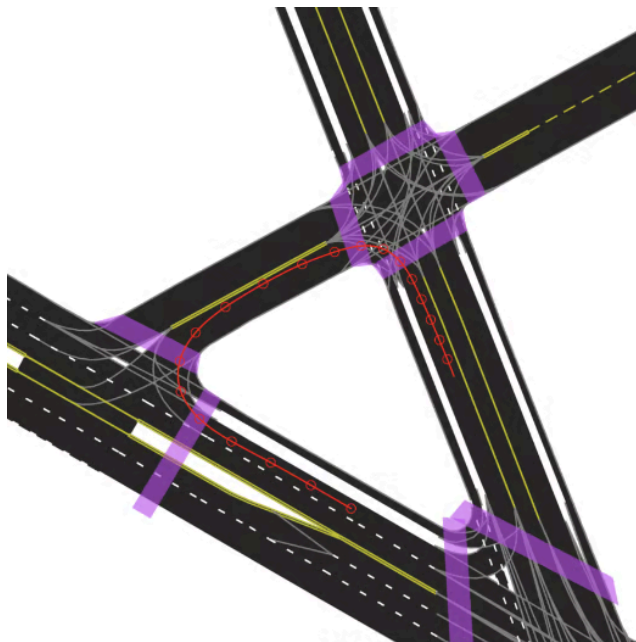


Changer le niveau d'abstraction et sélectionner un objet pour suivre son appartenance spatiale.

Cartes pour la conduite autonome

Construire une représentation géométrique et topologique exploitable par le véhicule.

Cartes pour la planification sur route



Un extrait de carte HD d'Argoverse 2.

Une carte haute définition, **HD**, décrit les voies, leurs limites, les traversées et les connexions utiles à la conduite.

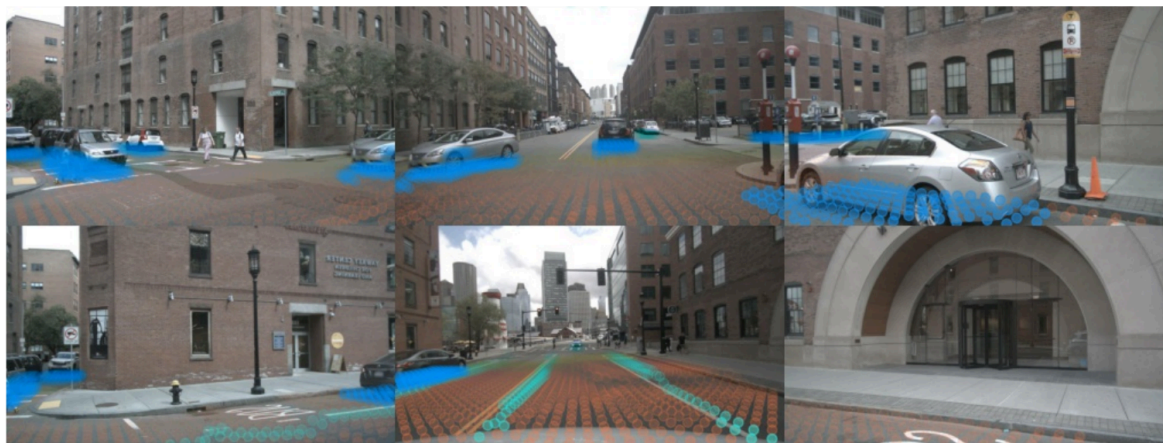
- **Blanc / jaune** : marquages et limites des voies.
- **Violet** : passages piétons.
- **Gris, aux intersections** : corridors implicites entre les voies.
- **Rouge** : trajectoire du véhicule, affichée comme repère.

Wilson et al., Argoverse 2, NeurIPS Datasets and Benchmarks 2021. Illustration du [guide officiel : Vector Map, Lane-Level Geometry](#).

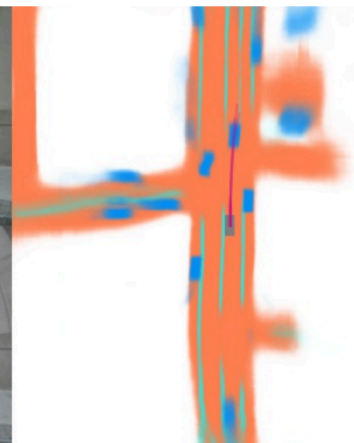
BEV: Vue à vol d'oiseau

La vue à vol d'oiseau, **BEV** (*bird's-eye view*), réunit les observations dans un même repère au sol, autour du véhicule.

Six caméras · prédictions reprojétées en points



Segmentation BEV



■ Véhicules ■ Surface carrossable ■ Voies

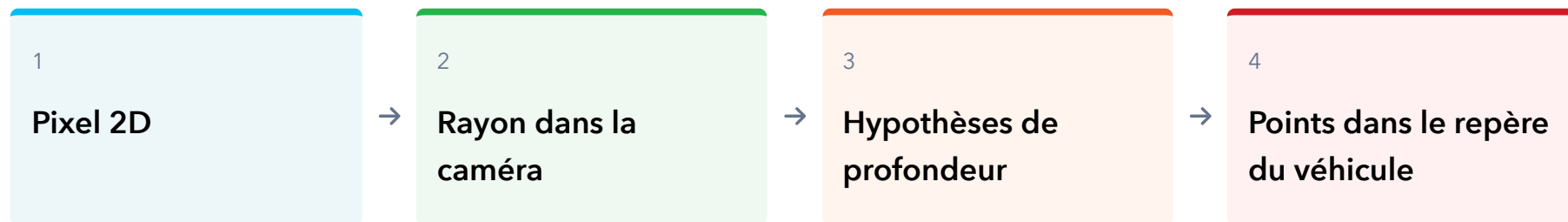
À droite, une **prédiction sémantique dans le plan du sol** : on peut comparer les positions et les distances autour du véhicule.

Phillon et Fidler, Lift-Splat-Shoot, ECCV 2020, figure 1. [Article et figure.](#)

BEV à partir d'image

Un pixel $\tilde{u} = (u, v, 1)^\top$ définit un rayon. Pour une profondeur axiale d :

$$X_C(d) = dK^{-1}\tilde{u}, \quad X_V(d) = R_{V \leftarrow C}X_C(d) + t_{V \leftarrow C}.$$



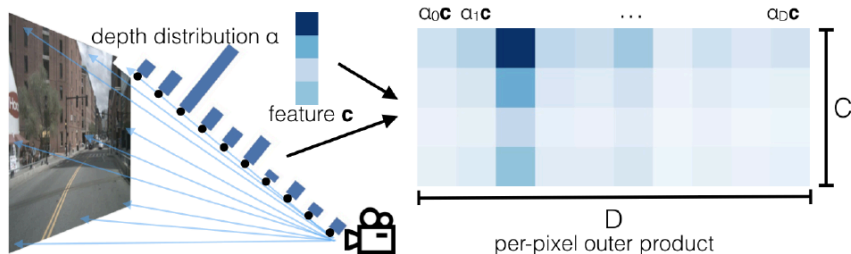
Ambiguïté

Sans profondeur ou hypothèse de surface, un pixel ne détermine pas un point 3D unique.

Si la voiture est équipée d'un LiDAR

On peut estimer la profondeur des pixels en les associant au nuage de points capté par le LiDAR.

Lift : distribuer la caractéristique dans la profondeur



Pour chaque pixel, le réseau prédit un **vecteur visuel** $f(u)$ et des poids de profondeur $p(d_k | u)$, dont la somme vaut 1.

$$F(u, d_k) = p(d_k | u) f(u).$$

Chaque profondeur reçoit une copie du vecteur, multipliée par son poids. La calibration détermine où placer cette contribution sur le rayon.

Exemple scalaire : $f(u) = 2$, avec des poids de 0,25 à 10 m et 0,75 à 20 m. Les contributions valent **0,5** et **1,5**.

Phillon et Fidler, ECCV 2020, Figure 3. [Article et source de la figure.](#)

Splat : accumuler dans les cellules au sol

Les points sont exprimés dans le repère véhicule. Pour chaque cellule c , on additionne **uniquement les contributions qui y tombent** :

$$\mathcal{I}_c = \{(u, k) \mid \text{cellule}(X_V(u, d_k)) = c\}, \quad F_{\text{BEV}}(c) = \sum_{(u, k) \in \mathcal{I}_c} p(d_k \mid u) f(u).$$

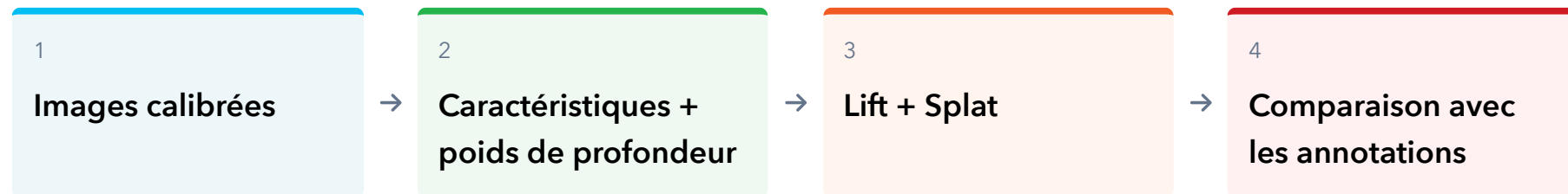
L'indice (u, k) désigne un pixel et une hypothèse de profondeur. Plusieurs pixels ou caméras peuvent contribuer à la même cellule.

Une grille de caractéristiques (features)

Les features ne sont pas des probabilités d'occupation. Par exemple, ils peuvent être des prédictions de classe (piétons, voitures, etc.).

Comment apprendre la représentation BEV ?

Dans Lift-Splat-Shoot, une tâche définie au sol peut superviser toute la chaîne :



Une cellule occupée par un véhicule

L'annotation indique « véhicule ». La tête BEV lui attribue une probabilité de 0,2 : sa perte d'entropie croisée vaut $-\ln(0,2) \approx 1,61$.

Cette erreur rétropropage vers les caractéristiques visuelles et les poids de profondeur qui ont contribué à la grille.

Les poids de profondeur peuvent ainsi être appris **sans annotations métriques de profondeur**. Ils ne sont pas nécessairement des mesures de profondeur calibrées.

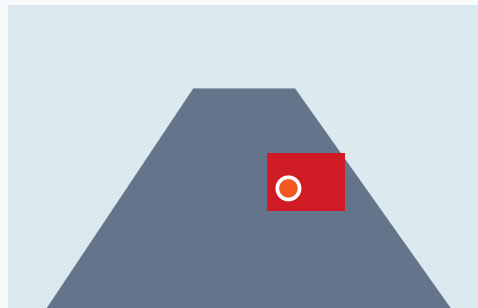
Phillion et Fidler, ECCV 2020. [Lift, Splat, Shoot](#).

Suivre un pixel jusqu'à la grille BEV

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

1. Image + caractéristique $f(u)$

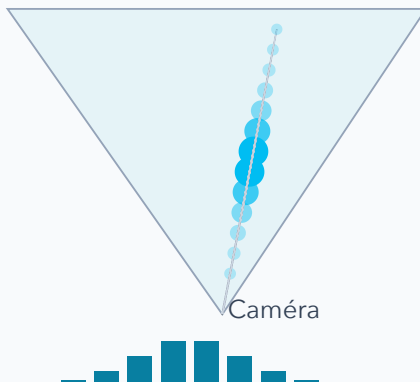


Pixel $u = 190, v = 120$

$K : f_x = f_y = 200 \text{ px}$

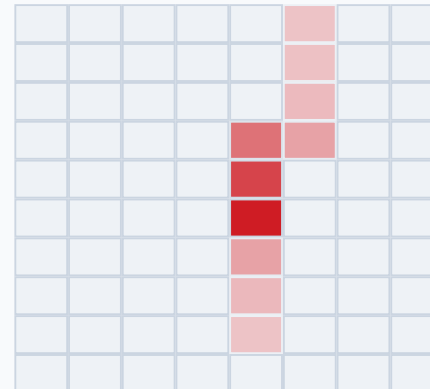
2. Distribuer sur les profondeurs

Point 3D = $d \cdot K^{-1} \tilde{u}$



Caméra

3. Accumuler dans la grille BEV



$\Sigma \text{ poids} = 1.000$

Pixel u Profondeur 15 m σ 3 m

Étape

Une caractéristique scalaire $f(u) = 1$ pour lire les poids ; dans le réseau, $f(u)$ est un vecteur.

Changer le pixel, la profondeur dominante et son incertitude. Suivre le déplacement et l'étalement dans la grille.

Quels éléments une carte doit-elle représenter ?

Élément	Géométrie possible	Information utile
Ligne / bordure	Polyligne	Position et séparation
Voie	Ligne centrale et limites	Largeur, direction, continuité
Passage piéton	Polygone ou limites	Zone de traversée
Intersection	Connexions entre voies	Mouvements autorisés



Une bordure sous forme de polyligne

Dans le repère véhicule (x vers l'avant, y vers la gauche) : $[(0, 2), (5, 2), (10, 3)]$ m, classe « bordure ». Les points décrivent son tracé ; la classe indique son sens pour le robot.

Construire une carte locale en ligne

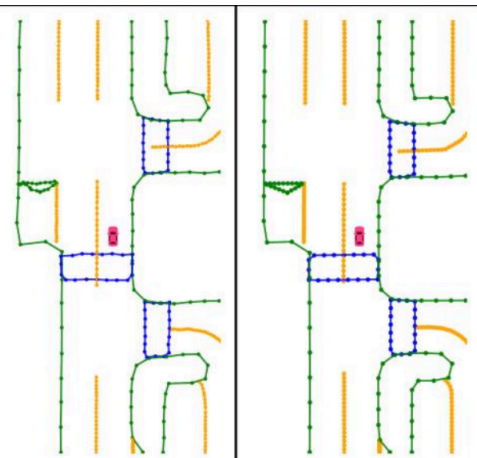
Le modèle estime les éléments visibles autour du véhicule à chaque instant. Le résultat peut compléter une carte existante ou fonctionner sans carte locale préétablie.

Six vues caméra de la même scène



Prédiction

Annotation

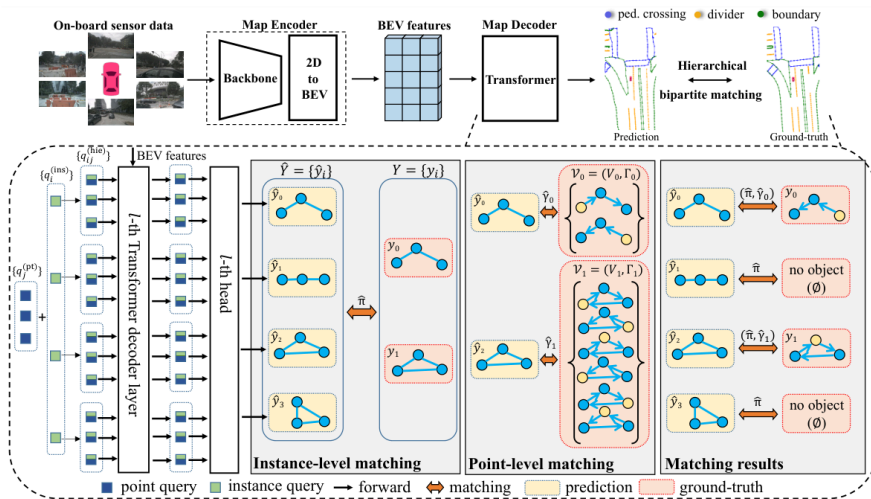


■ Passages piétons ■ Séparations de voies ■ Limites de chaussée

Pour l'entraînement: Il faut comparer les deux cartes. Les **points et lignes prédits** doivent retrouver la géométrie et la classe de chaque élément annoté.

Liao et al., MapTR, ICLR 2023, figure 1, première scène recadrée. [Article et figure.](#)

Exemple MapTR : prédire des éléments vectoriels



Le modèle prédit un ensemble d'éléments de carte, chacun décrit par des points.

L'apprentissage associe les prédictions aux annotations et tient compte des permutations valides des points. Puisqu'une même ligne lue dans les deux sens peut représenter la même géométrie.

Liao et al., MapTR, ICLR 2023, Figure 4. [Article et source de la figure.](#)

Définir la fonction de perte pour les éléments de carte

Après appariement des instances, comparer la classe et la géométrie. Pour les **points échantillonnés** des polygones prédites P et annotées Q :

$$d_{\text{Ch}}(P, Q) = \underbrace{\frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \|p - q\|}_{\text{prédiction vers annotation}} + \underbrace{\frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|q - p\|}_{\text{annotation vers prédiction}}.$$

Pour chaque point, chercher son **plus proche voisin**, puis moyenner les distances. Refaire le calcul dans l'autre sens.

Une bordure décalée de 20 cm

$P = \{(0;0), (1;0)\}$ et $Q = \{(0;0,2), (1;0,2)\}$ en mètres. Les voisins les plus proches sont tous à 0,2 m.

Les deux moyennes donnent $d_{\text{Ch}} = 0,2 + 0,2 = 0,4$ m.

Pourquoi comparer les polygones dans les deux sens ?

Une bordure réelle échantillonnée en $Q = \{0, 1, 2\}$ mètres, mais prédite seulement en $P = \{0, 1\}$: une partie manque.

Sens	Distances aux plus proches voisins	Moyenne
$P \rightarrow Q$	$0 \rightarrow 0 : 0 ; 1 \rightarrow 1 : 0$	0
$Q \rightarrow P$	$0 \rightarrow 0 : 0 ; 1 \rightarrow 1 : 0 ; 2 \rightarrow 1 : 1$	$1/3 \text{ m}$

La partie manquante impacte la fonction de perte

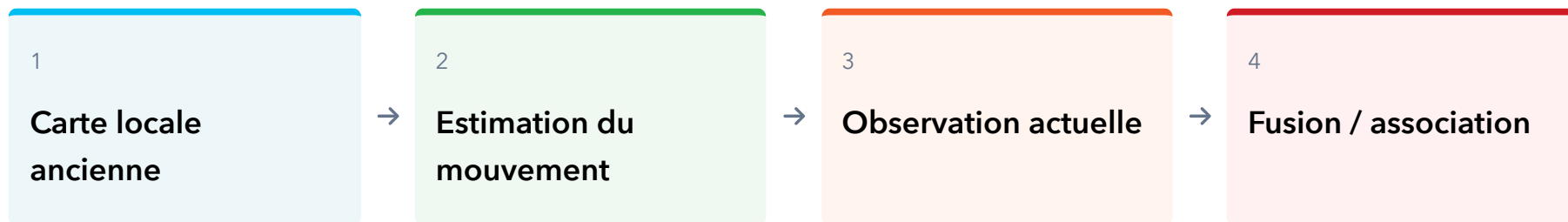
Tous les points prédits sont corrects : le premier sens ne pénalise rien. Le second détecte que le point à 2 m n'est pas couvert. Au total, $d_{\text{Ch}} = 1/3 \text{ m}$.

Donc, ça favorise la complétude des prédictions.

Aligner les observations successives

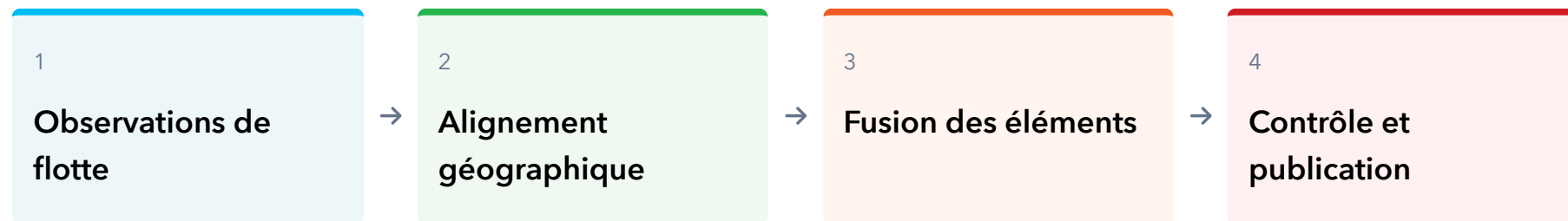
Une observation ancienne doit être exprimée dans le repère courant :

$$X_{V_t} = T_{V_t \leftarrow V_{t-1}} X_{V_{t-1}}.$$



Un point statique est à **10 m devant le véhicule**. Le véhicule avance de 2 m : dans le nouveau repère, ce point est à **8 m**. Sans estimation du mouvement, on fusionnerait à tort les positions 10 m et 8 m. Une erreur de recalage peut ainsi doubler une bordure.

Construire à l'échelle d'un réseau routier



Source	Apport	Limite à traiter
Carte de navigation / OpenStreetMap	Structure routière et connexions approximatives	Précision, complétude, actualité
Imagerie aérienne	Couverture et vue globale	Occultations, résolution, date
Capteurs de véhicules	Détails récents au niveau de la route	Couverture inégale, calibration
Correction humaine	Arbitrage des cas ambigus	Coût et traçabilité

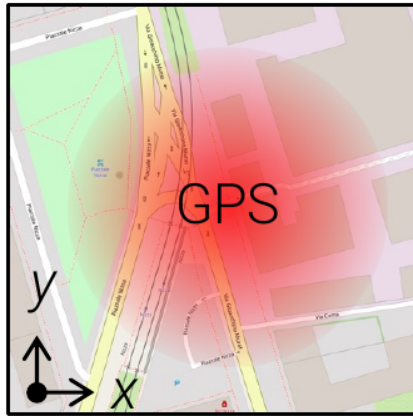
En pratique: Cartographier une nouvelle bretelle

La vue aérienne suggère le tracé ; plusieurs passages de véhicules mesurent les marquages et bordures. Après alignement, les observations sont fusionnées et un annotateur humain vérifie que la carte est correcte.

Utilisation de cartes publiques. Ex: OpenStreetMap (OSM)

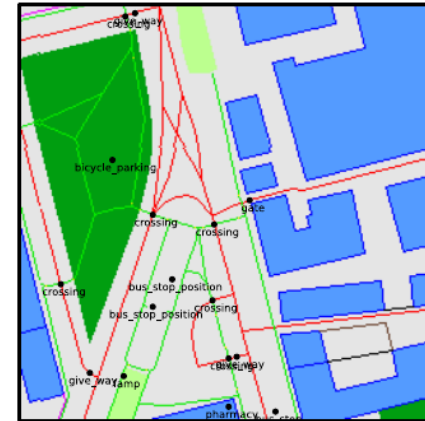
OSM, disponible gratuitement est une carte routière mondiale bâti à l'aide de données publiques et de crowdsourcing. Elle décrit notamment des routes et des bâtiments par des **lignes**, **polygones** et **attributs** et peut donc servir de base pour se localiser ou créer une carte HD.

Carte OSM et position GPS approximative



Rasteriser
(discrétiser
sur une
grille)
par classe

Raster sémantique

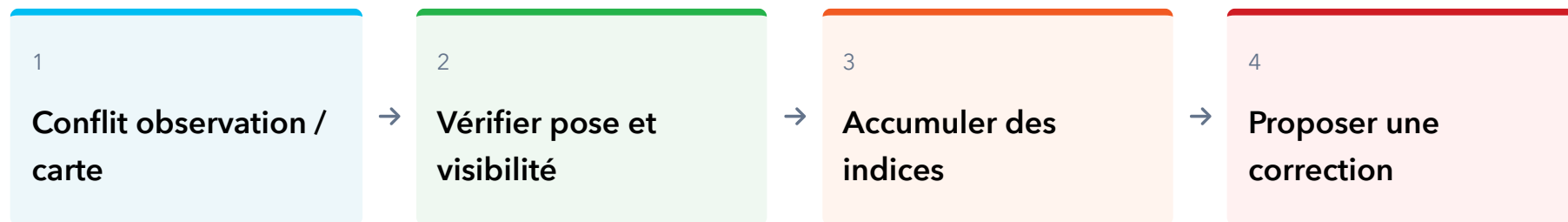


■ Bâtiments ■ Parcs ■ Herbe ■ Routes (lignes)

Il peut être utile de se servir de ces cartes comme base (imparfaite) pour la planification à long-terme et de les compléter avec les données des capteurs à bord du véhicule pour la planification à court-terme. Notamment pour détecter les changements, corriger la géométrie et détecter les objets dynamiques (piétons, voitures, etc.).

Sarlin et al., OrienterNet, CVPR 2023, figure 2, deux panneaux recadrés. [Article](#). Données © [OpenStreetMap](#) contributors.

Détecter un changement durable



Lorsqu'on observe l'environnement, on doit accumuler les indices avant de modifier une carte HD.

Une ligne absente peut simplement être occultée et donc ne pas être visible temporairement. Une ligne en apparence déplacée peut être l'effet d'une erreur de calibration.

Donc détecte un changement durable (permanent) dans la carte nécessite plusieurs indices cohérents. Par exemple, plusieurs passages de véhicule remarquant le même changement.

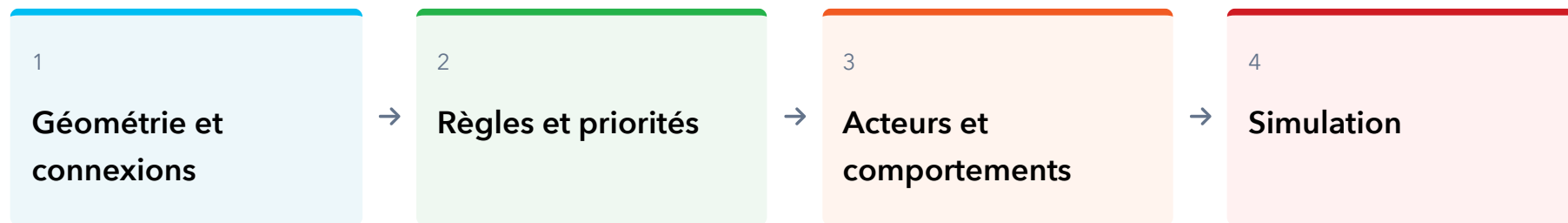
Traçabilité

Conserver date, provenance et incertitude. Une mise à jour validée doit pouvoir être distinguée d'une observation isolée.

Ça permet de bâtir des cartes HD très fiables pouvant être partagées par une flotte de véhicule.

Évidemment le processus est long et coûteux, d'où les déploiements graduels et limités de Waymo, Zoox, et autres compagnies de voitures autonomes.

Une carte abstraite supporte plusieurs scénarios



Les cartes qu'on construit fournissent un support spatial non seulement pour la navigation, mais aussi pour la simulation.

On peut les utiliser pour créer des scénarios en ajoutant des véhicules, des piétons, leurs états et leurs comportements. Une carte d'une même intersection permet de simuler un grand nombre de variations.

Abstraction

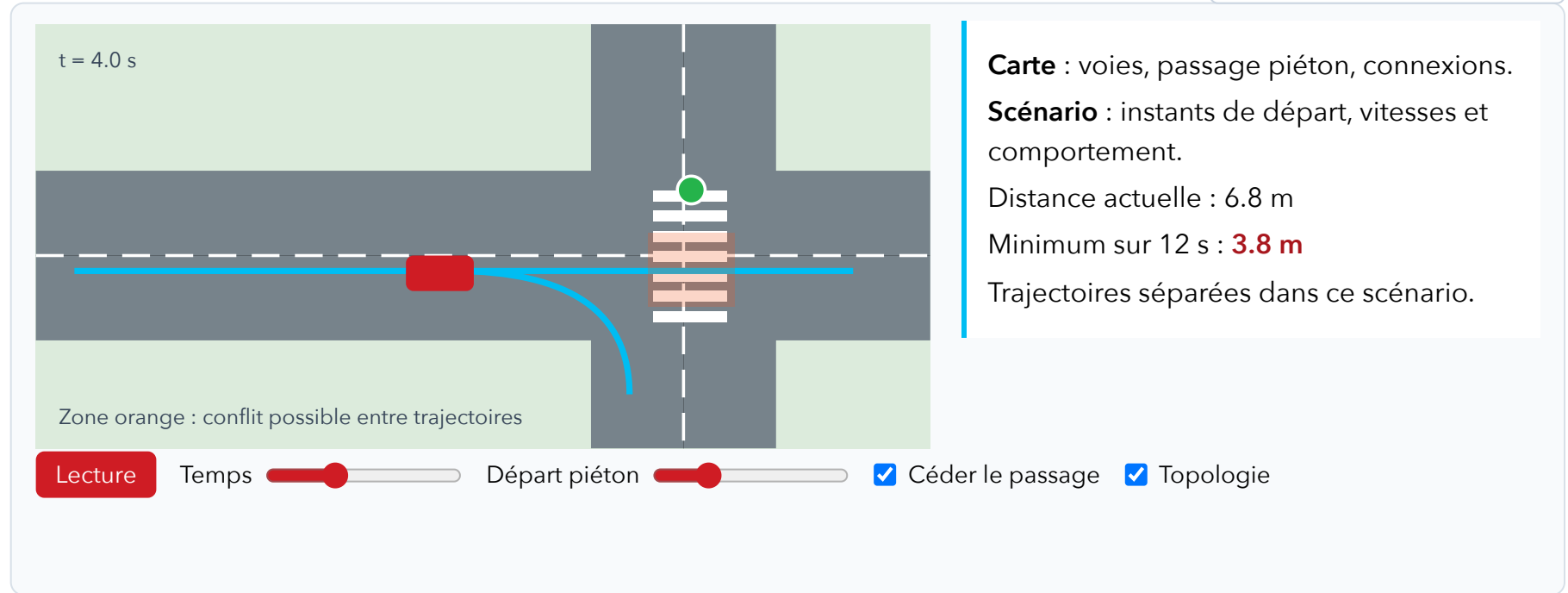
Le fait que la carte n'est qu'une collection d'objet, il est facile d'en ajouter ou retirer pour simuler des scénarios de conduite.

Beaucoup plus simple que de générer des données cohérentes et réalistes de caméras ou LiDAR pour les divers scénarios.

Explorer une intersection avec un piéton

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration



Faire varier l'instant de départ du piéton et la décision de céder le passage. Comparer les trajectoires sur une carte identique.

Supervision multimodale

Apprendre avec une source d'étiquettes différente du signal utilisé à l'inférence.

Les entrées et les étiquettes peuvent changer de modalité

Entrée du modèle	Source de supervision	Exemple de tâche
Image	Profondeur LiDAR	Estimation de profondeur
Image	Pose d'un modèle 3D	Estimation de pose d'objet
Image	Positions GNSS	Reconnaissance de lieux
Nuage LiDAR	Trajectoire géoréférencée	Reconnaissance de lieux 3D

Correspondance entre données

Les modalités doivent être synchronisées et correctement associées. Leur bruit se transmet aux étiquettes.

Exemple: Reconnaissance de lieux

Nous allons voir un exemple: la reconnaissance de lieux.

L'objectif est de répliquer la capacité de relocalisation d'un GPS à l'aide seulement d'images ou de scans LiDAR. Le modèle appris peut ensuite être utilisé lorsque les signaux GNSS (GPS) ne sont plus disponibles.

Retrouver un lieu déjà vu auparavant

Une image requête est comparée à une base d'images de référence.

Étant donné que les images sont prises à des moments différents. Un défi est de retrouver un lieu malgré un changement d'éclairage, de conditions météo ou d'occultations.

Le résultat est: une **association candidate** (paire d'images), accompagnée d'un score (confiance/invertitude).



(a) Mobile phone query

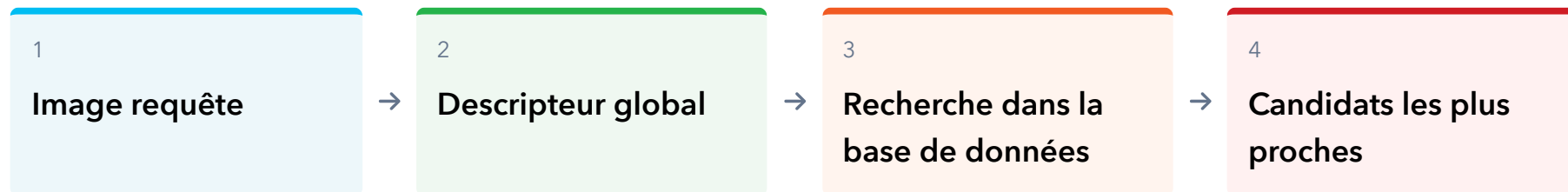


(b) Retrieved image of same place

Arandjelović et al., NetVLAD, CVPR 2016, Figure 1. Article et source de la figure.

La reconnaissance de lieux est une recherche

$$z_q = f_\theta(I_q), \quad z_j = f_\theta(I_j), \quad j^* = \arg \min_j d(z_q, z_j).$$



Le descripteur global est un vecteur compact associé à l'image entière. Les vecteurs de la base sont calculés et indexés à l'avance.

Ambiguïté visuelle

Deux lieux distincts peuvent se ressembler.
Conserver plusieurs candidats avant de vérifier la géométrie.

Un descripteur global résume les caractéristiques locales

Un encodeur produit un vecteur local $f_i \in \mathbb{R}^D$ à chaque pixel ou patch dans l'image. L'agrégation résume ces vecteurs en **un descripteur pour l'image**.



Le descripteur conserve les informations apprises pour reconnaître le lieu. Ses valeurs ne sont pas des coordonnées géographiques.

De la recherche à la fermeture de boucle



Une fermeture de boucle relie deux poses d'un même graphe lorsqu'un lieu est revisité. La vérification géométrique fournit une transformation relative mesurée et son incertitude.

Échelle de la mesure

Une paire d'images monoculaires fournit généralement une translation à une échelle inconnue. Une contrainte métrique complète exige une information supplémentaire.

Exemple: Un robot revient devant l'entrée du laboratoire

Le robot retrouve une ancienne image de l'entrée. Des correspondances RGB-D vérifiées permettent d'estimer le déplacement relatif entre les deux prises de vue et d'ajouter la contrainte au graphe.

Résidu d'une fermeture de boucle

$$r_{ij} = \text{Log}(Z_{ij}^{-1}T_i^{-1}T_j) \in \mathbb{R}^6.$$

Quantité	Lecture
Z_{ij}	Pose mesurée du repère j vers le repère i
$T_i = T_{W \leftarrow i}, T_j = T_{W \leftarrow j}$	Poses des deux caméras dans le monde
$T_i^{-1}T_j$ et Z_{ij}	Deux transformations du repère j vers le repère i
$Z_{ij}^{-1}T_i^{-1}T_j$	Identité si prédiction et mesure coïncident
$\text{Log}(\cdot)$	Convertit le désaccord en six composantes locales

Ajouter une fermeture de boucle au graphe

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

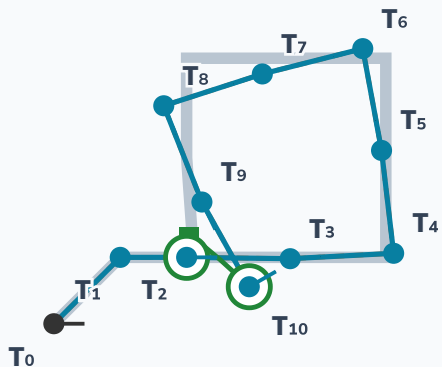
Reconnaissance de lieux

Bonne association · $T_{10} \leftrightarrow T_2$

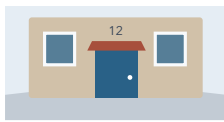


Optimiser

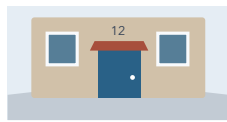
Graphe non optimisé



T_{10} retrouve T_2 ?



Requête : entrée A



Référence : entrée A

Même lieu, deux passages.

Écart au trajet réel

0,95 → 0,95 m

Coût du graphe

844,2 → 844,2

RMSE des positions · simulation SE(2)

Fermeture fortement pondérée · perte quadratique

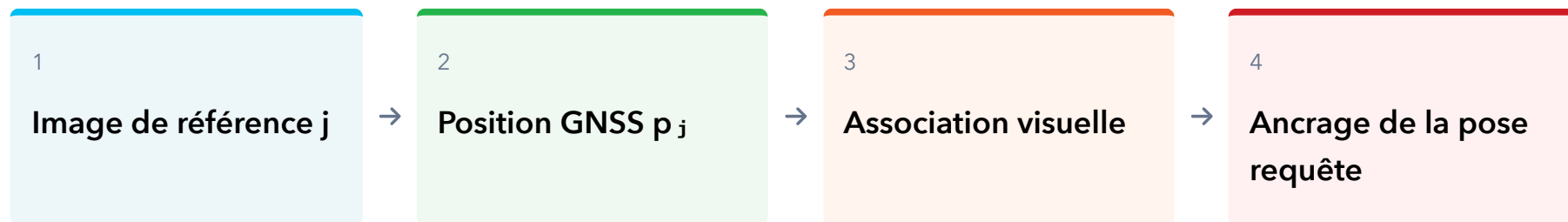
T_0 fixé · orientations indiquées par les traits · même échelle dans les deux cas

— Trajet réel — Odométrie initiale — Poses estimées — Fermeture $T_2 \leftrightarrow T_{10}$

Même entrée A : la bonne fermeture est prête. Cliquer sur Optimiser.

Choisir une bonne ou une mauvaise association, puis cliquer sur « Optimiser » et observer la trajectoire.

Utiliser la position GNSS d'une image retrouvée



Une base géolocalisée fournit une position approximative du lieu reconnu, même si la requête ne dispose pas de GNSS.

Ancrage approximatif

La requête et la référence peuvent être prises à plusieurs mètres l'une de l'autre. Cette différence s'ajoute au bruit du GNSS.

Localiser un robot sans réception GNSS

Une image du robot reconnaît une façade présente dans une base de données géolocalisée. La position de la référence donne un voisinage plausible.

Résidu d'un ancrage géographique approximatif

Avec t_i la position de la caméra requête dans un repère métrique commun :

$$r_i^{\text{geo}} = t_i - p_j^{\text{GNSS}}, \quad E_i^{\text{geo}} = (r_i^{\text{geo}})^\top \Sigma_{\text{geo}}^{-1} r_i^{\text{geo}}.$$

Ce facteur suppose que les positions des deux prises de vue sont suffisamment proches. Son incertitude doit couvrir l'erreur GNSS, le décalage de prise de vue et les défauts d'association.

Un ancrage dans un repère local, en mètres

Si $t_i = (12, 5, 0)$ et $p_j = (10, 4, 0)$, alors $r_i^{\text{geo}} = (2, 1, 0)$ m. Avec $\Sigma_{\text{geo}} = 4I \text{ m}^2$, le coût vaut $(2^2 + 1^2)/4 = 1,25$.

Comment créer un jeu de données d'entraînement pour la reconnaissance de lieux

D'abord, on peut sélectionner des paires positives et négatives à partir des positions GNSS.

Pour des images géolocalisées en p_i et p_j :

$$\|p_i - p_j\| < r_+ \Rightarrow \text{positif possible}, \quad \|p_i - p_j\| > r_- \Rightarrow \text{négatif possible}, \quad r_- > r_+.$$

Une zone intermédiaire est ignorée pour réduire les ambiguïtés. La proximité spatiale donne une supervision faible : les images peuvent regarder dans des directions opposées.

Bruit d'étiquetage

Bruit GNSS, rues parallèles, façades répétitives et changements de point de vue produisent des paires trompeuses.

Rayons illustratifs : 10 m et 25 m

À 6 m : positif possible. À 18 m : paire ignorée. À 40 m : négatif possible. Deux images à 6 m peuvent toutefois regarder des façades opposées ; la proximité seule ne suffit pas.

Négatifs faciles, difficiles et faux négatifs

Type	Distance géographique	Distance descripteurs	Effet
Facile	Lieux distincts	Grande	Perte souvent nulle
Difficile	Lieux distincts	Petite	Signal d'apprentissage utile
Faux négatif	Même lieu ou recouvrement réel	Petite	Signal d'apprentissage erroné

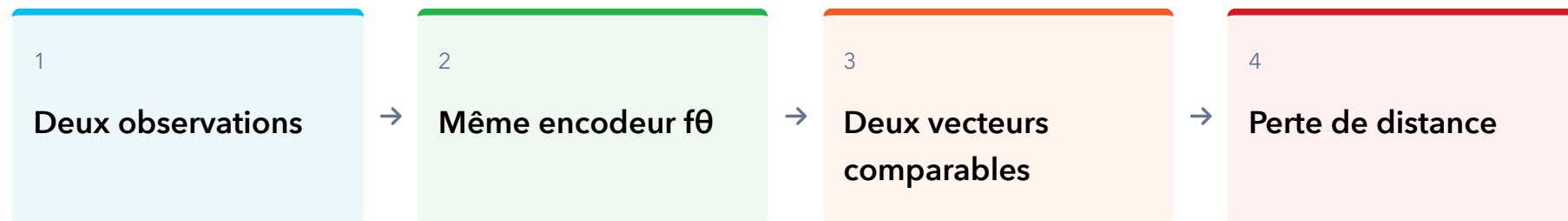


Requête : l'entrée d'un stationnement

Négatif facile : une forêt. **Négatif difficile** : l'entrée semblable d'un autre stationnement. **Faux négatif** : la même entrée, géolocalisée trop loin à cause d'une erreur GNSS.

Comment entraîner: Un réseau siamois

$$z_a = f_{\theta}(I_a), \quad z_b = f_{\theta}(I_b).$$



Les deux branches utilisent les mêmes paramètres. La perte apprend une géométrie où les exemples compatibles sont proches.

Le même carrefour, de jour et de nuit

Chaque image traverse le même encodeur. La paire positive encourage des descripteurs proches malgré l'éclairage ; les gradients des deux branches mettent à jour les mêmes poids.

Perte contrastive : apprendre avec des paires

Distance entre descripteurs : $D = \|z_a - z_b\|_2$. Avec $y = 1$ pour un positif et une marge $m > 0$:

$$\ell_{\text{paire}} = \begin{cases} D^2 & y = 1 \quad (\text{même lieu}), \\ \max(0, m - D)^2 & y = 0 \quad (\text{lieux différents}). \end{cases}$$

Paire, avec $m = 1$	Distance D	Perte	Effet recherché
Carrefour de jour / même carrefour de nuit	0,4	$0,4^2 = 0,16$	Rapprocher les descripteurs
Deux carrefours différents	0,4	$(1 - 0,4)^2 = 0,36$	Les séparer
Deux carrefours différents	1,2	0	Déjà assez éloignés

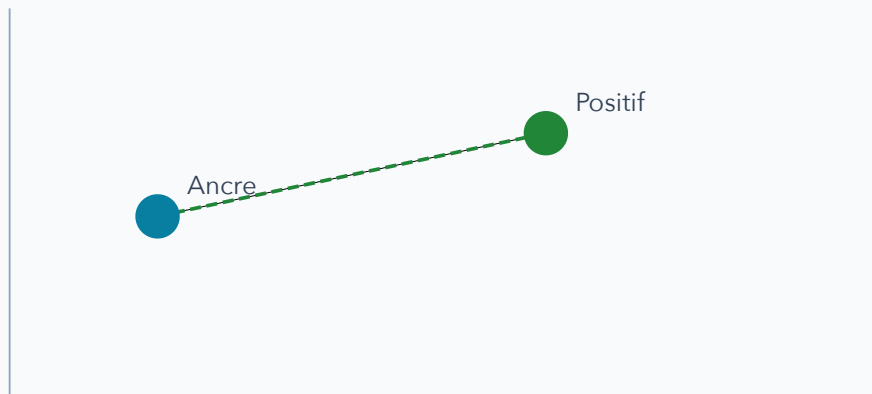
La marge fixe une séparation suffisante

Les négatifs au-delà de m ne contribuent plus à cette perte. D mesure un écart dans l'espace des descripteurs, pas une distance en mètres entre lieux.

Rapprocher les positifs, séparer les négatifs dans l'espace des descripteurs

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration



Paire positive

$d = 2.184$

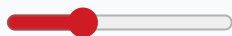
$\ell = 4.770$

Rapprocher les deux représentations.

Déplacer les points, puis appliquer un pas de gradient.

Pas de gradient

Marge



1.0



Paire positive

0 mise(s) à jour

Changer le type de paire et la marge, puis appliquer des pas de gradient.

Perte triplet : imposer une marge relative

Ancre a : carrefour de nuit ; positif p : même carrefour de jour ; négatif n : un autre carrefour.

$$\ell_{\text{tri}} = \max \left(0, \underbrace{\|z_a - z_p\|_2}_{d_{ap}} - \underbrace{\|z_a - z_n\|_2}_{d_{an}} + m \right).$$

On veut $d_{an} \geq d_{ap} + m$: le négatif doit être plus éloigné que le positif **d'au moins la marge**.

$d_{ap} = 0,4, m = 0,2$

Distance négative d_{an}

Perte

Négatif plus loin, mais marge insuffisante

0,5

$0,4 - 0,5 + 0,2 = 0,1$

Marge satisfaite

0,7

$\max(0, -0,1) = 0$

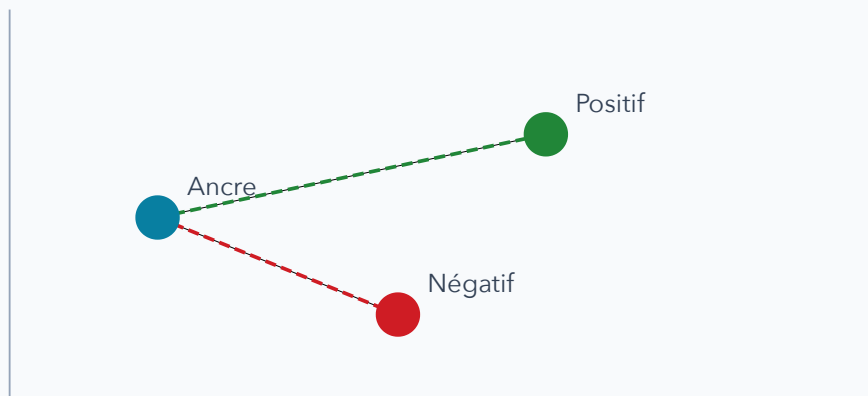
Avantage

La fonction de perte triplet est plus stable. On évite de rapprocher puis éloigner les descripteurs successivement. On fait les deux en même temps.

Voir l'effet de la perte triplet

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration



Déplacer les points, puis appliquer un pas de gradient.

Pas de gradient

Marge



1.0

0 mise(s) à jour

$$d(a,p) = 2.184$$

$$d(a,n) = 1.476$$

$$\ell = \max(0, d(a,p) - d(a,n) + m)$$

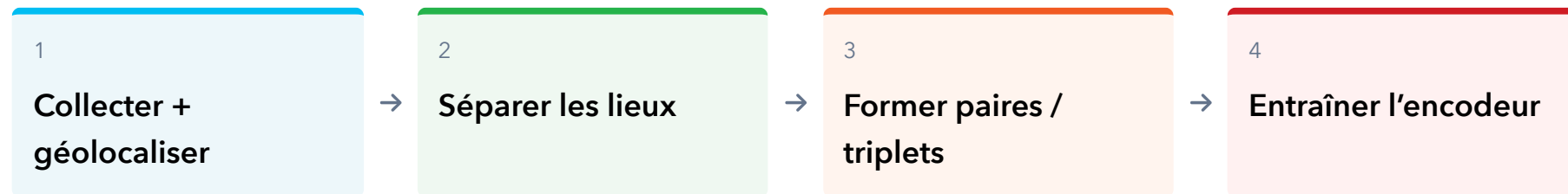
$$\ell = 1.708$$

La contrainte de marge est violée.

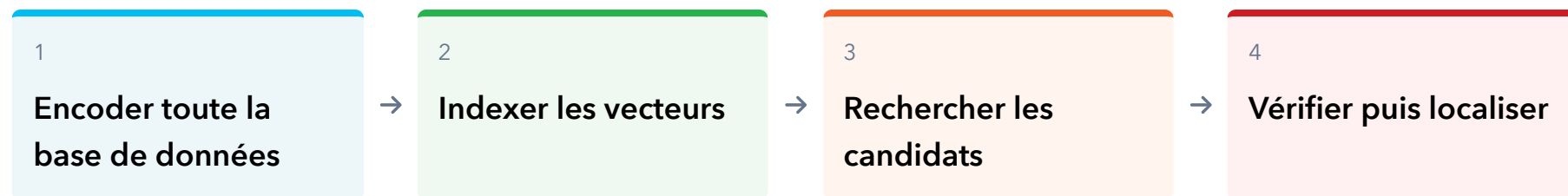
Déplacer ancre, positif et négatif. Modifier la marge, puis suivre les mises à jour des trois points.

Le pipeline d'apprentissage et de recherche

Pipeline: d'apprentissage:



Pipeline: de recherche:



Les conditions visuelles peuvent être très variées



Jour/nuit, météo, saisons, structure et point de vue modifient l'apparence.

Une bonne évaluation doit séparer ces facteurs et inclure des exemples réellement différents des données d'entraînement.

Warburg et al., Mapillary SLS, CVPR 2020, Figure 2. [Article et source de la figure.](#)

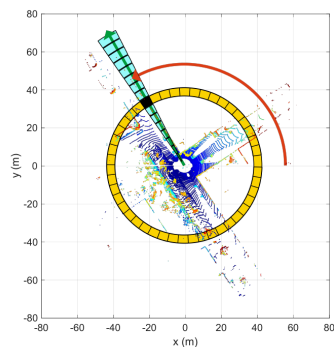
Apprendre des robustesses ciblées

Variation	Approches concrètes	Évaluation
Saisons	Positifs multitemporels ; caractéristiques locales stables	Revisites été/hiver indépendantes
Météo	Collectes diversifiées ; augmentations photométriques et dégradations plausibles	Vraies séquences pluie/brouillard
Heure du jour	Couples jour/nuit ; entraînement avec fortes variations d'éclairage	Requêtes nocturnes réservées
Point de vue	Positifs multivues, EigenPlaces ; vérification géométrique locale	Déplacements latéraux et vues opposées

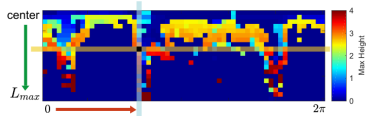
Limite

Une augmentation synthétique aide à couvrir une variation ; elle ne remplace pas l'évaluation sur des observations réelles.

Scan Context : Reconnaissance de lieux avec LiDAR



(a) Bin division along azimuthal and radial directions



(b) Scan context

Le plan horizontal est divisé en anneaux et secteurs.

Chaque cellule conserve une hauteur maximale, produisant une matrice qui décrit la structure autour du capteur.

Le descripteur original est construit explicitement, sans entraînement supervisé.

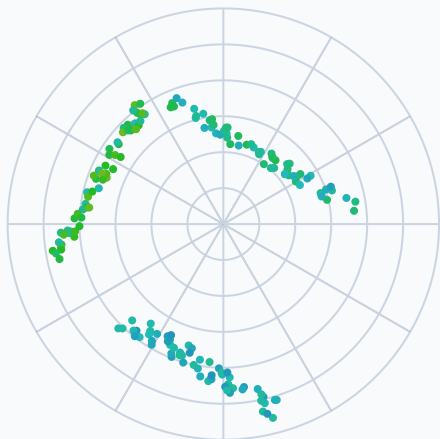
Kim et Kim, Scan Context, IROS 2018, Figure 1. [Article et source de la figure.](#)

Construire et aligner un descripteur polaire

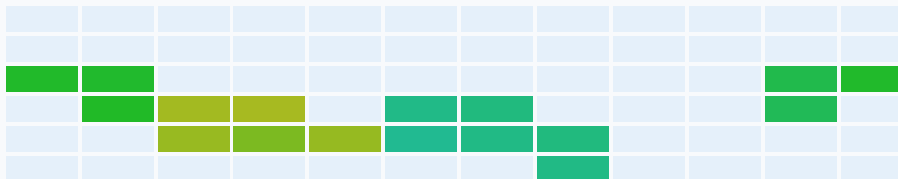
Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

Nuage et secteurs polaires

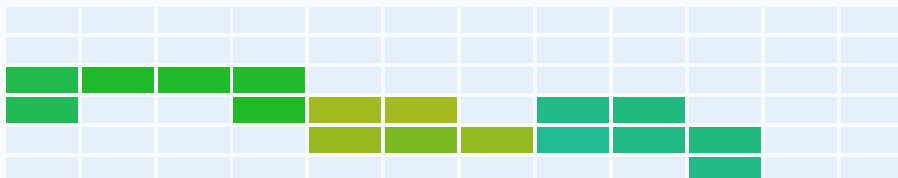


Référence : hauteur maximale / cellule



Colonnes : secteurs · lignes : anneaux

Requête après décalage circulaire

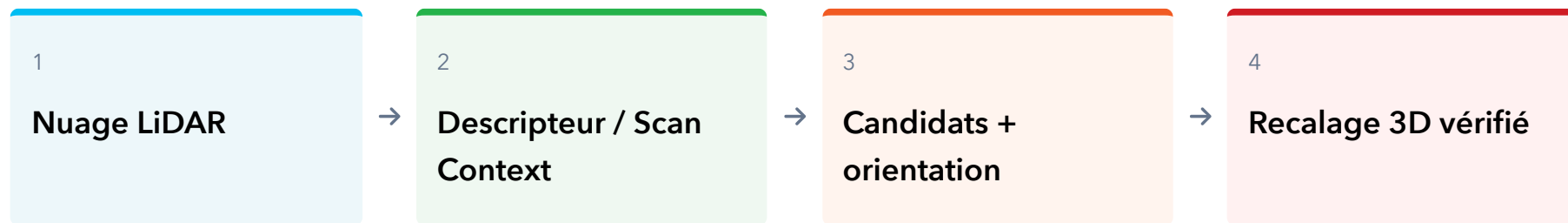


Rotation LiDAR 60° Décalage 0 **Aligner** Distance = **0.395**

Une rotation devient un décalage de colonnes. La translation et les occultations modifient aussi le contenu des cellules.

Faire tourner le nuage, puis rechercher le décalage de colonnes qui minimise la distance.

Reconnaissance LiDAR : du candidat à la contrainte



Le LiDAR apporte une géométrie moins dépendante de l'éclairage. Les changements de structure, les occultations et les déplacements latéraux restent difficiles.

Synthèse

Relier supervision, représentation, géométrie et décision robotique.

Concevoir un système supervisé

Question	Exemples du cours
Quelle cible est réellement disponible ?	Classe, boîte, pose, polyligne, position GNSS
Quelle représentation sert la tâche ?	Probabilités, volume d'objet, carte, descripteur
Quelle perte exprime l'erreur pertinente ?	Entropie croisée, régression, géométrie, marge
Quelle vérification protège l'usage robotique ?	Validation indépendante, visibilité, géométrie, incertitude

Garbage In Garbage Out

En apprentissage supervisé, le nerf de la guerre est la qualité des données d'entraînement. Si on n'a pas de bonnes données, on obtiendra de mauvais résultats.